

TaskOrbit Conversational Agent

Design Documentation

Sprint 13 — Final Release Consolidated Documentation

Generated: 2026-07-12

Source: GitHub Wiki ([amos2026ss04-taskorbit-conversational-agent.wiki](#))

Built by: AMOS SS2026 Team 04, in collaboration with Meisterwerk App GmbH, Berlin

Table of Contents

1. [Project Overview](#)
 2. [Design Documentation](#)
 3. [System Architecture](#)
 4. [Architecture — Runtime Components](#)
 5. [Architecture — Code Components](#)
 6. [Architecture Decisions](#)
 7. [Tech Stack Decision](#)
 8. [Basic Agent Runtime](#)
 9. [Agent Routing Foundation](#)
 10. [Transitive Workflow Dependencies](#)
 11. [Audio Input via LiveKit — STT](#)
 12. [Audio Output via LiveKit — TTS](#)
 13. [Voice Interruption for Audio Chat](#)
 14. [Voice Call UI & Audio Pipeline Updates](#)
 15. [Component Benchmarking](#)
 16. [Logging & Observability](#)
-

Project Overview

Project Overview

The Problem — The “Giant Prompt” Trap

As you add more features to a voice agent, the system instructions get longer and harder to manage. Giant prompts lead to **instruction drift** — the AI forgets its role or gets confused mid-conversation. The result is a slow, unreliable experience that feels “lost” during real-time speech.

Traditional voice agents fail due to three compounding issues:

- **Giant Prompt Trap** — unmanageable monolithic instructions
- **Scaling Chaos** — no clean way to add new tasks without breaking existing ones
- **Latency & Reliability** — a bloated prompt slows every single LLM call

The Solution — Modular Orchestration

TaskOrbit replaces the single giant prompt with a **clean, modular framework**. The agent’s “brain” is broken into structured **Task Definitions** — JSON files that describe exactly what an agent should do, in what order, with what tools.

Three core principles:

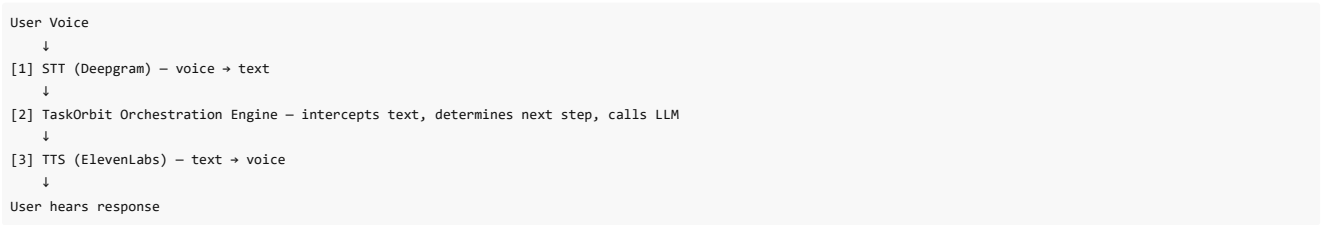
JSON-Based Orchestration — structured Task Definitions replace messy prompts. Each task defines its own workflow steps, required inputs, agent persona, and tool calls.

Dynamic Task Switching — a flexible orchestration engine manages conversation flow, activating the right prompt at the right moment based on the user’s progress.

Reliable Execution — by enforcing strict boundaries and “flight paths”, the agent stays grounded, completes tasks in sequence (Step A → Step B → Step C), and triggers tools with precision.

The Pipeline

At runtime, a TaskOrbit conversation flows through three stages:



The orchestration engine is the only stateful component. Everything else — STT, LLM, TTS — is stateless and interchangeable.

Key Concepts

Term	Meaning
Task Definition	A JSON file describing one complete agent workflow (e.g. "book an appointment")
Workflow Step	A single action within a task (e.g. "greet", "collect data", "confirm", "end call")
Agent	A configured identity — persona, instruction, pipeline config, tools — defined in the JSON
Session	One conversation run, holding the current step index and collected data
Orchestration Engine	The Python component that reads the task JSON and drives the conversation forward step by step
Tool	A callable action the agent can trigger (e.g. extract data, end call, transfer to another agent)

Team

Built by **AMOS SS2026 Team 04** in collaboration with **Meisterwerk App GmbH**, Berlin.

Role	Contact
CTO / Industry Partner	Andreas Zink — andi@meisterwerk.app
Software Engineer / Industry Partner	Preet Vadaliya — preet@meisterwerk.app

See also: [Architecture — Runtime](#) · [Agent Task Schema](#)

Design Documentation

TaskOrbit Design Documentation

System Overview

TaskOrbit is a real-time voice AI platform built around a modular orchestration engine. Rather than encoding agent behaviour in a monolithic system prompt, the system uses JSON-based Task Definitions to describe each agent's workflow, required inputs, and available tools. At runtime, a Python orchestration engine drives each conversation step by step, loading only the context relevant to the current moment. This prevents instruction drift as conversations grow longer, and makes agent behaviour auditable, testable, and changeable without modifying application code.

Runtime Architecture

The system runs as two separate backend processes:

- FastAPI HTTP API (port 8000)** — handles REST requests: conversation turns, agent configuration management, LiveKit token minting, TTS synthesis, and health checks. Stateless per request.
 - LiveKit Voice Worker** — a long-running process that joins LiveKit rooms and runs the full voice pipeline: STT via Deepgram, orchestration, LLM inference, TTS via ElevenLabs. Audio flows over WebRTC — the HTTP API never handles audio bytes. Browser (React + LiveKit JS SDK) | ↔ HTTP/JSON ↔ FastAPI backend (:8000) | ↔ REST: conversations, agent configs, tokens, TTS
- | ↔ WebRTC audio ↔ LiveKit Cloud ↔ Voice Worker ↔ Deepgram STT ↔ Orchestration Engine ↔ LLM (OpenAI / Gemini) ↔ ElevenLabs TTS

The orchestration engine is the only stateful component. It holds session state — current workflow step, collected data, conversation history — and decides what context to pass to the LLM at each turn.

Code Structure

backend/src/taskorbit/ ↔ api/ HTTP routes (conversations, livekit, tts, agent configs) ↔ orchestration/ ConversationOrchestrator — the core engine ↔ agents/ BaseAgent, SalesAgent, TechnicalSupportAgent, AgentRegistry ↔ tools/ data_extraction, end_call, agent_transfer ↔ integrations/llm/ LLMClient protocol, OpenAI and Gemini clients, factory ↔ livekit_agent/ AgentSession, OrchestratorAgent, voice worker entry point ↔ database/ SQLAlchemy models, repositories, Alembic migrations ↔ logging/ structlog configuration ↔ observability/ Prometheus metrics, Grafana dashboards, Loki log aggregation ↔ config.py Centralised settings via pydantic-settings -- types.py Shared Pydantic domain types

api/ — FastAPI routes. The conversations route persists messages automatically via ConversationOrchestrator._persist_messages(). The agent configs route provides full CRUD for saved agent configurations backed by PostgreSQL.

orchestration/ — ConversationOrchestrator.process_message() is the main entry point. It detects intent, selects an agent, selects the active tool, builds a minimal system prompt, calls the LLM, and returns a ConversationResponse.

integrations/llm/ — Provider-agnostic LLMClient Protocol. Concrete clients: OpenAIClient (openai SDK) and GeminiClient (google-genai SDK). A factory function get_llm_client() reads agent_config.llm.provider — provider selection is per-task, not global.

livekit_agent/ — AgentSession wires Deepgram STT, ElevenLabs TTS, and Silero VAD. OrchestratorAgent overrides llm_node() to route inference through ConversationOrchestrator instead of the LLM plugin directly. Push-to-talk is enforced via a reply_requested flag — the session only generates a reply when explicitly triggered.

database/ — SQLAlchemy async ORM. Tables: conversations, conversation_messages, tool_executions, users, agent_configurations. Alembic manages migrations.

observability/ — Prometheus metrics at /metrics (port 8000) and /metrics (port 8001 for the worker). Loki aggregates structured logs from both processes. Grafana dashboards are auto-provisioned.

Agent Task Schema

Every agent is described by a single JSON file conforming to agent-task.schema.json (JSON Schema draft 2020-12, version 1.0.0). Three top-level fields are schema_version, task, and agent.

The task block defines: name, description, required_inputs (fields the agent must collect, each with name, type, required, description), and workflow_steps (ordered steps with id, action, and an optional tool reference).

The agent block defines: agent_id, metadata (display_name, persona), instruction (the system prompt), first_message, variables (session-scoped with defaults), pipeline (STT/LLM/TTS provider config), and tools (callable actions with typed parameters).

Reserved workflow step actions: send_first_message returns agent.first_message directly without an LLM call; end_call terminates the session. All other actions delegate to the LLM with the current conversation history.

Technology Stack

Area	Technology	Version	Role
Frontend	React + Vite	19 / 5	Web UI and LiveKit client
Backend	Python + FastAPI	3.11 / latest	HTTP API and orchestration
Voice	LiveKit + LiveKit Agents	latest	WebRTC audio transport and agent session management
STT	Deepgram (nova-3)	latest	Speech-to-text
LLM	OpenAI or Google Gemini	gpt-4o-mini / gemini-2.5-flash	Reasoning and response generation
TTS	ElevenLabs (eleven_multilingual_v2)	latest	Text-to-speech, 29 languages
Database	PostgreSQL 16	16	Persistent storage
ORM	SQLAlchemy (async) + Alembic	2.x	ORM and migrations
Deployment	Google Cloud Platform	—	Cloud Run, Cloud SQL, Artifact Registry
Containerisation	Docker + docker-compose	—	Local and production containers
Logging	structlog + Loki + Grafana	—	Structured logging and observability

Key Architecture Decisions

JSON-based task orchestration over monolithic prompts — Each agent workflow is described by a versioned JSON file rather than embedded in code or a giant prompt. This makes agent behaviour auditable, testable, and changeable without touching application code. The orchestration engine loads only the context for the current workflow step, which prevents instruction drift as conversations progress.

Python as the backend language — LiveKit's Python Agents SDK is the primary supported path for building voice agent workers; the Node.js SDK is less mature for this use case. Pydantic v2 validates JSON task definitions directly against Python models, eliminating a separate validation library. The LLM and STT/TTS ecosystems are predominantly Python-first.

Per-task LLM provider selection — The LLM provider and model are declared in each agent's task definition rather than set globally. This allows different agents in the same deployment to use different providers or models. Adding a new provider requires writing one client class and one factory branch — no orchestration changes.

Two-process backend — The HTTP API (FastAPI) and the voice worker (LiveKit Agents) run as separate processes with independent lifecycles. The API is stateless and scales horizontally; the worker holds a persistent LiveKit connection and scales independently. Merging them would couple two incompatible execution models.

SQLAlchemy async ORM with Alembic migrations — The database layer uses SQLAlchemy with the async psycopg driver. Schema changes are managed via Alembic migrations. Conversation messages, tool executions, and agent configurations are all persisted in PostgreSQL. Tests use in-memory SQLite via the same ORM layer without requiring a running database.

Structured logging across both processes — Both the HTTP API and the voice worker emit JSON-structured logs to stdout via structlog. Promtail ships logs to Loki; Grafana queries both processes in a single dashboard. Token usage and pipeline latency are tracked across both the REST path and the voice path using Loki LogQL over the `llm_call_completed` event.

System Architecture

Taskorbit Conversational Agent System Architecture

Issue: [#74 Documentation](#)

This page describes the real, current architecture of TaskOrbit as implemented in `backend/src/taskorbit` . It replaces the earlier draft, which had unresolved merge markers and described components that were never built or have since been restructured. Every claim below is verified against the code.

1. Overview

TaskOrbit is a modular, real-time conversational agent platform that supports both **voice** and **text** conversations through the same core pipeline.

Voice path (`livekit_agent/session.py`):

```
user audio → Silero VAD → STT → OrchestratorAgent (llm_node) → TTS → audio out
```

The LiveKit worker (`worker.py`) joins a LiveKit room, and `livekit_agent/session.py` builds an `AgentSession` wired with VAD, STT, TTS, and `livekit_agent/llm.py` 's `OrchestratorAgent` — a thin bridge that hands each user turn to the same `ConversationOrchestrator` used by the text path.

Text path: the API (`api/routes/conversations.py`) calls `ConversationOrchestrator` directly — no LiveKit round trip. Voice and text conversations are therefore driven by identical orchestration logic; only the audio I/O differs.

2. Backend Structure

The diagrams referenced in earlier drafts of this page do not match the current code (see [\\$7 Diagrams](#)). The structure below is the source of truth.

Flat top-level modules

Module	Role
<code>config.py</code>	Centralised configuration — a single <code>Pydantic Settings(BaseSettings)</code> class. Not <code>config/settings.py</code> .
<code>constants.py</code>	Shared constants (e.g. <code>DEV_USER_EMAIL</code>).
<code>types.py</code>	Core dataclasses/enums shared across the backend (<code>AgentConfig</code> , <code>ConversationRequest</code> , <code>ConversationResponse</code> , <code>LLMProvider</code> , <code>STTProvider</code> , <code>TTSProvider</code> , etc.).
<code>worker.py</code>	LiveKit worker endpoint (poetry run taskorbit-worker dev) — connects to LiveKit, dispatches a session per room.
<code>workflow_rules.py</code>	Conditional workflow rule evaluation (#71).
<code>agent_config_util.py</code>	Agent configuration helpers.
<code>cli.py</code>	CLI entrypoints.

Packages

Package	Contents
<code>agents/</code>	<code>BaseAgent</code> and concrete agents (<code>SalesAgent</code> , <code>TechnicalSupportAgent</code> , <code>GeneralInquiryAgent</code> , <code>AppointmentManagementAgent</code> , ...) plus <code>AgentRegistry</code> , the single constructor for all agent types.
<code>orchestration/</code>	A single <code>ConversationOrchestrator</code> class in <code>__init__.py</code>. This is the core of TaskOrbit — the sole entry point for processing a user message (<code>process_message</code> / <code>process_message_stream</code>), covering intent detection, agent selection, workflow dependency resolution, handoff enforcement, slot extraction, system prompt construction, and the LLM call. There is no <code>models.py</code> , <code>engine.py</code> , <code>task_loader.py</code> , <code>agent_router.py</code> , <code>planner.py</code> , or <code>handoff_manager.py</code> — earlier docs describing a multi-file orchestration package were aspirational and never built; this correction supersedes them.
<code>api/routes/</code>	<code>agent_configs.py</code> , <code>conversations.py</code> , <code>livekit.py</code> , <code>tts.py</code> , <code>user_agents.py</code> .
<code>integrations/llm/</code>	<code>factory.py</code> , <code>base.py</code> , <code>errors.py</code> , <code>messages.py</code> , <code>prompts.py</code> , <code>scope_check.py</code> , plus four provider clients — see \$3 .
<code>tools/</code>	<code>data_extraction.py</code> , <code>end_call.py</code> , <code>agent_transfer.py</code> , <code>generic_api.py</code> .
<code>livekit_agent/</code>	<code>llm.py</code> (<code>OrchestratorAgent</code> , the bridge into <code>ConversationOrchestrator</code>), <code>session.py</code> (builds the <code>AgentSession</code> with STT/TTS/VAD wiring).
<code>database/</code>	<code>crud.py</code> , <code>models.py</code> , <code>seed_data.py</code> .
<code>logging/</code>	Structured logging setup (<code>setup.py</code>).
<code>observability/</code>	<code>metrics.py</code> , <code>middleware.py</code> , <code>tracing.py</code> .
<code>intent/</code>	LLM-driven intent routing.
<code>language/</code>	Language/locale handling.
<code>slots/</code>	<code>extractor.py</code> , <code>models.py</code> — slot extraction for data-collection tasks.

3. LLM Provider Layer

TaskOrbit supports **four** LLM providers, not two. The provider set is defined by the `LLMProvider` enum (`types.py`) and routed through `integrations/llm/factory.py` :

Provider	Client	Selected via
OpenAI	<code>integrations/llm/openai_client.py</code>	<code>LLMProvider.OPENAI</code>
Google (Gemini)	<code>integrations/llm/gemini_client.py</code>	<code>LLMProvider.GOOGLE</code>
OpenRouter	<code>integrations/llm/openrouter_client.py</code>	<code>LLMProvider.OPENROUTER</code>
Ollama (self-hosted)	<code>integrations/llm/ollama_client.py</code>	<code>LLMProvider.OLLAMA</code>

`get_llm_client()` returns the concrete client for the provider named in a given `AgentConfig.llm`, so the provider is selected **per-agent**, not globally. Each client is lazy-imported (an `ImportError` becomes a clear `LLMConfigError` rather than a crash at module load), and a required API key/URL is validated up front per provider.

A settings-driven guard, `_guard_provider_model_match` (#99), rejects a mismatched provider/model pair before it reaches the concrete client — e.g. a `gemini-*` model name selected under the `openai` provider fails fast with a clear error instead of returning a 404 mid conversation. OpenRouter model names are exempt from this check since they're free-form namespaced strings (e.g. `qwen/qwen-2.5-7b-instruct:free`) that don't collide with the other providers' prefixes.

4. STT / TTS

Deepgram and ElevenLabs are both **dual-capable**: either provider can serve as STT or TTS, selected independently per agent (#135, `STTProvider` / `TTSProvider` enums in `types.py`). This corrects the earlier fixed assumption of Deepgram-for-STT / ElevenLabs-for-TTS — that pairing is only the default when no per-agent configuration is present.

`livekit_agent/session.py` also pins provider-native fallback models (`scribe_v2_realtime` for ElevenLabs STT, `aura-2-andromeda-en` for Deepgram TTS) so a saved config referencing a model name from the “wrong” provider can't crash the voice pipeline at runtime.

5. Key Systems

Issue	System	What it does	Status
#71	Conditional workflow engine	Turn-1 entry-agent locking, <code>workflow_rules</code> -driven prerequisite branching (<code>workflow_rules.py</code>), and <code>allowed_handoffs</code> gating. Decides whether a handoff is allowed.	Live on main
#212	Agent handoff resolution	<code>resolve_transfer_target()</code> (<code>tools/agent_transfer.py</code>) replaces fragile regex-based target normalization, so an approved handoff's target resolves reliably across both text and voice paths.	In-flight on <code>fix/212-agent-handoff-resolution</code> ; documented here as part of the final release
#68	Component benchmarking	<code>_build_pipeline_latency_ms()</code> stamps <code>latency_ms (llm_call / tool_call / total)</code> onto every <code>ConversationResponse</code> . (PR #184)	Live on main
#168	Persona constraints	<code>default_persona_constraints()</code> (<code>types.py</code>) is the baseline guardrail applied to any agent without its own constraints, enforced at three pipeline stages: <code>pre_intent</code> , <code>pre_extraction</code> , and <code>pre_llm</code> .	Live on main

In short: **#71** decides whether an agent is allowed to hand a conversation off; **#212** makes sure that once a handoff is allowed, it lands on the correct target agent, reliably, on both text and voice.

6. Configuration

All configuration is centralised in the flat `taskorbit/config.py`, a single Pydantic `Settings(BaseSettings)` class loaded once from environment variables / `.env` and accessed everywhere via `get_settings()`.

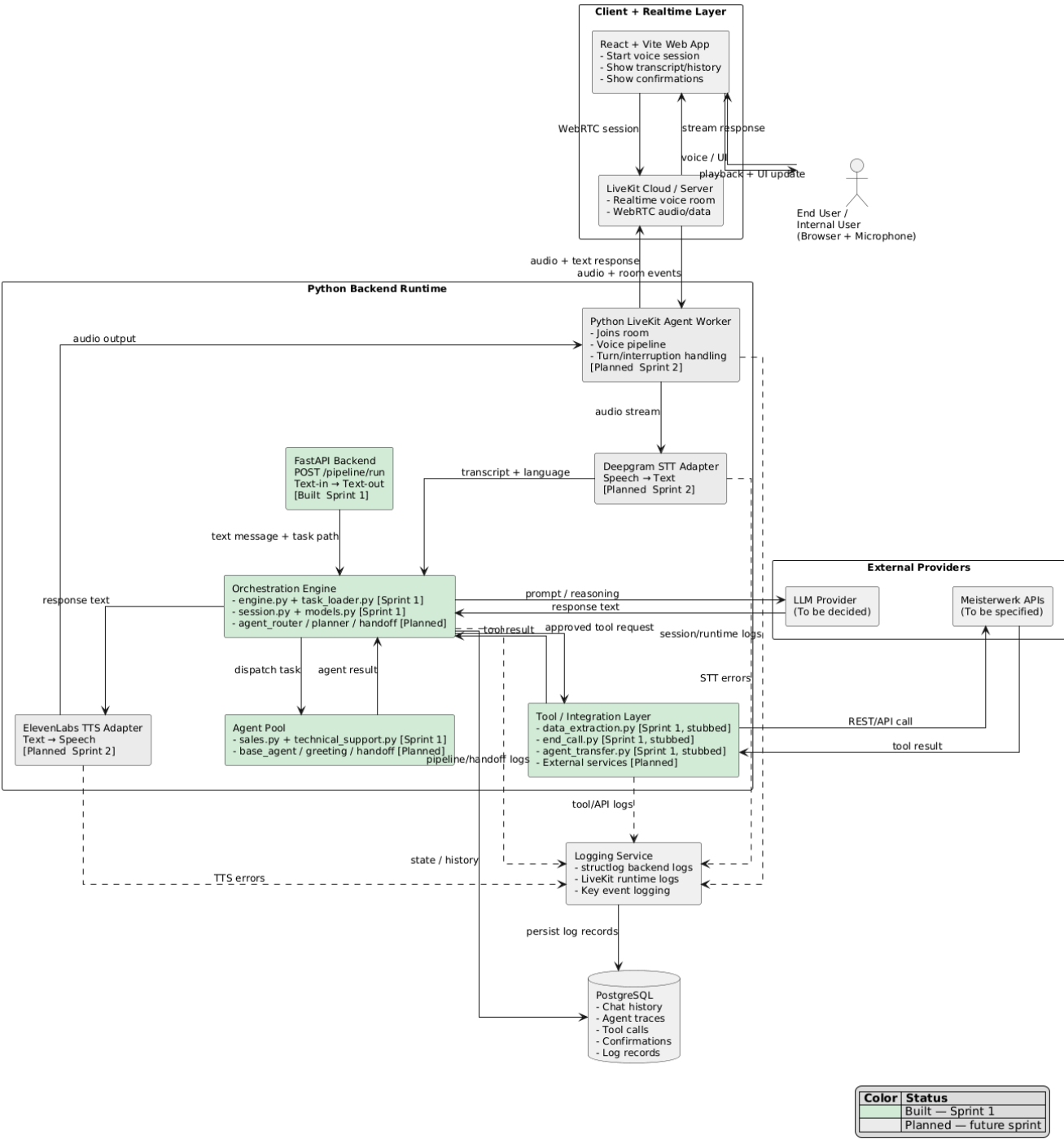
Area	Fields
Application	<code>app_env</code> , <code>log_level</code> , <code>api_host</code> , <code>api_port</code> , <code>cors_allow_origins</code>
Database	<code>database_url</code>
LiveKit	<code>livekit_url</code> , <code>livekit_api_key</code> , <code>livekit_api_secret</code> , <code>livekit_agent_dispatch_name</code> , <code>livekit_agent_name</code>
Deepgram	<code>deepgram_api_key</code> , <code>deepgram_model</code> , <code>deepgram_language</code>
ElevenLabs	<code>elevenlabs_api_key</code> , <code>elevenlabs_voice_id</code> , <code>elevenlabs_model</code>
LLM providers	<code>openai_api_key</code> , <code>google_api_key</code> , <code>openrouter_api_key</code> , <code>ollama_base_url</code> (plus per-provider default model fields)
LLM behaviour	<code>llm_timeout_seconds</code> , <code>enable_scope_shortcircuit</code> (feature flag gating the pre-LLM out-of-scope check; prompt-only guardrails remain on regardless)
Observability	<code>metrics_enabled</code> , <code>otel_enabled</code> , <code>otel_exporter_otlp_endpoint</code> , <code>log_json</code>

7. Diagrams

🚧 **TODO — pending regeneration.** The two diagrams below are stale: they (and the wiki pages describing them) reference files that no longer exist, e.g. `orchestration/models.py`, `orchestration/handoff_manager.py`, `api/routes/session_routes.py`. They are kept here as placeholders only and must **not** be treated as accurate until regenerated from the real structure documented in [§2](#). Do not cite them as current in other docs until this note is removed.

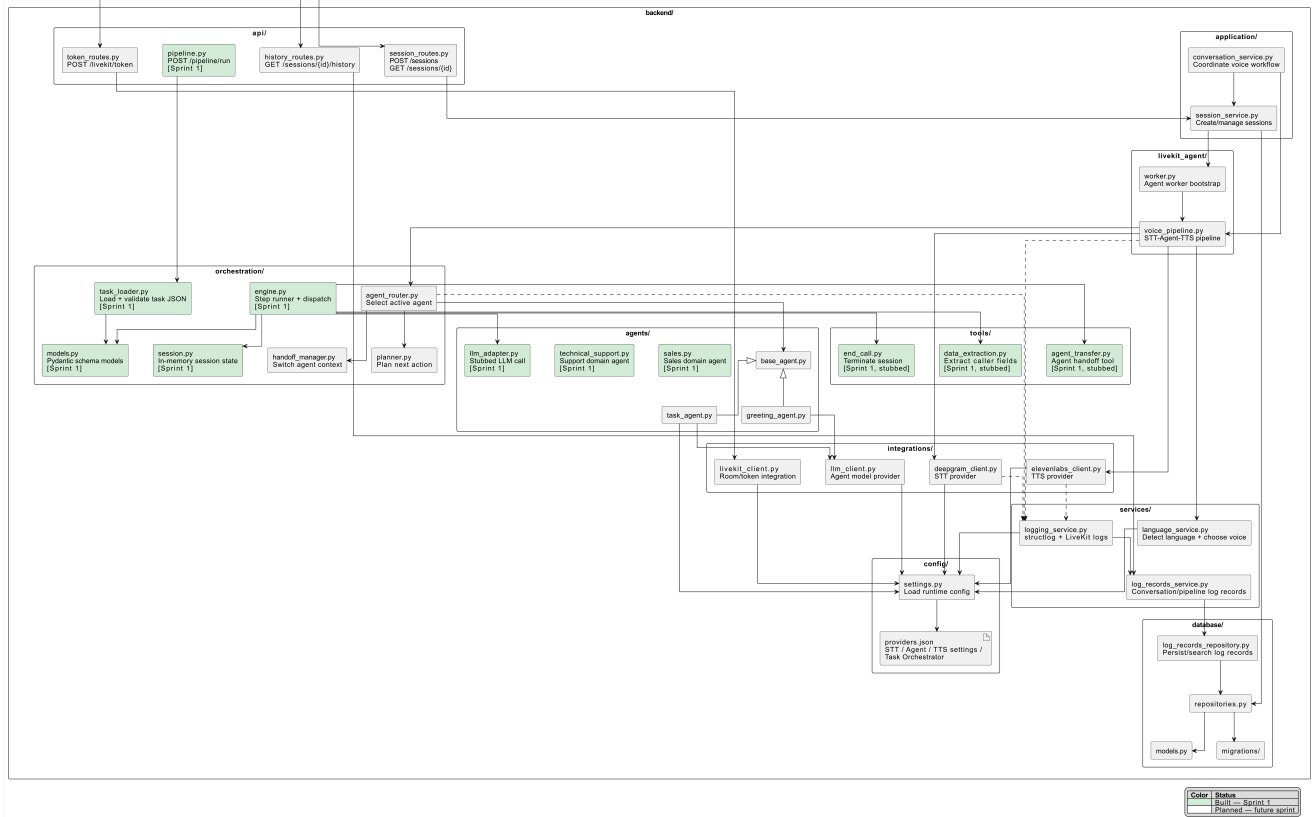
Runtime Components Overview (stale — pending regeneration)

Runtime Components Overview — Architecture v2



Code Components Overview (stale — pending regeneration)

Code Components Overview — Architecture v2 (Sprint 1)



8. Further Reading

Page	What's inside
LLM Integration	Developer flow for the provider factory
Agent Routing Foundation	Intent routing pipeline details
Agent Persona Guardrails	Persona constraint enforcement
Component Benchmarking	#68 latency/reliability benchmarking
LiveKit Cloud Setup	Configure LiveKit credentials
API Reference	All current endpoints
Getting Started	Run the project locally

Architecture — Runtime Components

Architecture — Runtime Components

Issue: [#74 Documentation](#)

This page describes how the live TaskOrbit system flows end-to-end at runtime. It replaces an earlier draft written when voice was a “Sprint 2+” future feature and the text pipeline was a stubbed placeholder — both are now live, and the voice path is the primary production path. See [System-Architecture](#) for the pipeline overview and [Architecture — Code Components](#) for the file structure referenced below.

Diagram file: `Documentation/Runtime Components Overview.png` — **stale, pending regeneration.** It reflects the structure this page used to describe. Do not treat it as accurate until it's regenerated from the component map below.

Component Map

Client + Realtime Layer

Component	Technology	Responsibility
Web App	React + Vite	Starts a voice or text session, shows transcript, confirmations, and handoff state
Realtime Layer	LiveKit Cloud	WebRTC audio transport between browser and the backend worker

Python Backend Runtime

Component	File(s)	Responsibility
LiveKit Agent Worker	<code>worker.py</code>	Joins a LiveKit room per session, dispatches the voice pipeline
Voice Session Builder	<code>livekit_agent/session.py</code>	Wires Silero VAD, STT, TTS, and the orchestrator bridge into a LiveKit AgentSession
Orchestrator Bridge	<code>livekit_agent/llm.py</code> (OrchestratorAgent)	Hands each voice turn to ConversationOrchestrator
Text API entry point	<code>api/routes/conversations.py</code>	Calls ConversationOrchestrator directly for text conversations
Conversation Orchestrator	<code>orchestration/__init__.py</code> (ConversationOrchestrator)	Single stateful core: intent detection, agent selection, workflow dependency resolution, handoff enforcement, slot extraction, LLM call, tool dispatch
Agent Pool	<code>agents/__init__.py</code>	Domain agent implementations (sales, technical support, general inquiry, appointment management, ...) via AgentRegistry
Tool Layer	<code>tools/</code>	<code>data_extraction</code> , <code>end_call</code> , <code>agent_transfer</code> (handoff resolution, #212), <code>generic_api</code> (config-driven external calls, #66)
LLM Provider Factory	<code>integrations/llm/factory.py</code>	Selects one of four providers per agent — see below
Logging / Observability	<code>logging/setup.py</code> , <code>observability/</code>	Structured logging, metrics, tracing

External Providers

Component	Providers	Notes
LLM	OpenAI, Google (Gemini), OpenRouter, Ollama	Selected per agent via <code>AgentConfig.llm.provider</code> ; not hardcoded to a single provider. <code>_guard_provider_model_match</code> (#99) rejects mismatched provider/model pairs before the call.
STT / TTS	Deepgram, ElevenLabs	Both are dual-capable — either provider can serve STT or TTS, selected independently per agent (#135).
Database	PostgreSQL	Conversation state, agent configs, seed data (<code>database/</code>)

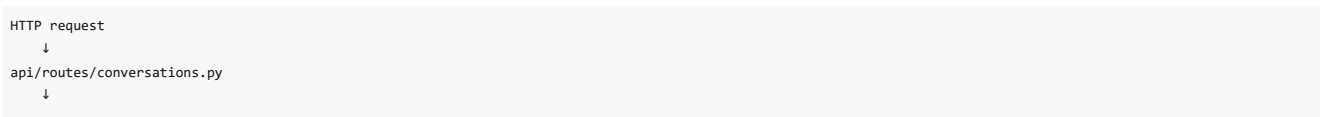
Live Data Flow

Both the voice and text paths converge on the same `ConversationOrchestrator` ; only the audio I/O differs.

Voice path (primary production path)



Text path



```
orchestration/__init__.py – ConversationOrchestrator.process_message[_stream]
↓
JSON response – ConversationResponse (reply, intent, agent, slots, status, latency_ms)
```

ConversationOrchestrator.process_message covers, per turn: manual-transfer short-circuit → end-call detection → scope refusal checks (#168, three stages) → intent detection → agent selection (turn-1 entry-agent lock + allowed_handoffs gating, #71) → workflow dependency resolution (workflow_rules.py , #71) → slot extraction → system prompt construction → LLM call → tool dispatch, including handoff target resolution via resolve_transfer_target() (#212).

Key Systems in the Runtime Path

Issue	System	Runtime effect
#71	Conditional workflow engine	Decides whether the current turn is allowed to hand off — turn-1 locking, workflow_rules prerequisite branching, allowed_handoffs gating
#212	Agent handoff resolution	Once a handoff is allowed, resolves its target reliably on both voice and text via resolve_transfer_target()
#68	Component benchmarking	_build_pipeline_latency_ms() stamps latency_ms (llm_call / tool_call / total) onto every ConversationResponse, on both paths (PR #184)
#168	Persona constraints	default_persona_constraints() enforced at pre_intent, pre_extraction, and pre_llm — refuses off-topic turns before they reach the LLM

Key Design Principle

ConversationOrchestrator is the **only stateful component** driving a turn — STT, the LLM call, and TTS are stateless operations invoked by it. This means:

- STT/TTS providers can be swapped per agent (Deepgram ↔ ElevenLabs) without touching the orchestrator
- The LLM provider can be swapped per agent (OpenAI, Gemini, OpenRouter, Ollama) via AgentConfig.llm.provider , without touching orchestration code
- Voice and text conversations share one code path, so a fix or feature in the orchestrator applies to both automatically

See also: [System Architecture](#) · [Architecture — Code Components](#) · [Component Benchmarking](#)

Architecture — Code Components

Architecture — Code Components

Issue: [#74 Documentation](#)

This page maps the file and package structure of `backend/src/taskorbit`. It replaces an earlier draft written against an early sprint plan; a number of the files and packages it described (`orchestration/models.py`, `engine.py`, `agents/llm_adapter.py`, `application/`, `services/`, `config/settings.py`, among others) were never built or were superseded during implementation. Every path below is verified against the current code. See [System-Architecture](#) for the authoritative pipeline overview and cross-cutting systems (LLM providers, STT/TTS, key issues).

Diagram file: `Documentation/Code Components Overview.png` — **stale, pending regeneration**. It was drawn against the structure this page used to describe and references files (`orchestration/models.py`, `orchestration/handoff_manager.py`, `api/routes/session_routes.py`, ...) that don't exist. Do not treat it as accurate until it's regenerated from the tree below.

Backend Package Structure

```
backend/
└─ src/
   └─ taskorbit/
      │ config.py           - Settings(BaseSettings), flat, not config/settings.py
      │ constants.py        - shared constants (e.g. DEV_USER_EMAIL)
      │ types.py            - AgentConfig, ConversationRequest/Response, provider enums
      │ worker.py           - LiveKit worker endpoint
      │ workflow_rules.py   - conditional workflow rule evaluation (#71)
      │ agent_config_util.py - agent configuration helpers
      │ cli.py              - CLI entrypoints
      │ agents/
      │ │ └─ __init__.py    - BaseAgent, concrete agents, AgentRegistry
      │ │
      │ │ └─ orchestration/
      │ │ │ └─ __init__.py  - ConversationOrchestrator (single class)
      │ │ │
      │ │ │ └─ api/
      │ │ │ │ └─ routes/
      │ │ │ │ │ └─ agent_configs.py
      │ │ │ │ │ └─ conversations.py
      │ │ │ │ │ └─ livekit.py
      │ │ │ │ │ └─ tts.py
      │ │ │ │ │ └─ user_agents.py
      │ │ │
      │ │ │ └─ integrations/
      │ │ │ │ └─ llm/
      │ │ │ │ │ └─ factory.py      - get_llm_client(), provider/model guard (#99)
      │ │ │ │ │ └─ base.py         - LLMClient protocol
      │ │ │ │ │ └─ errors.py       - LLMConfigError, LLMError, LLMTimeoutError
      │ │ │ │ │ └─ messages.py
      │ │ │ │ │ └─ prompts.py
      │ │ │ │ │ └─ scope_check.py  - is_message_in_scope (#168)
      │ │ │ │ │ └─ openai_client.py
      │ │ │ │ │ └─ gemini_client.py
      │ │ │ │ │ └─ openrouter_client.py
      │ │ │ │ │ └─ ollama_client.py
      │ │ │
      │ │ │ └─ tools/
      │ │ │ │ └─ data_extraction.py
      │ │ │ │ └─ end_call.py
      │ │ │ │ └─ agent_transfer.py  - resolve_transfer_target() (#212)
      │ │ │ │ └─ generic_api.py     - config-driven external API adapter (#66)
      │ │ │
      │ │ │ └─ livekit_agent/
      │ │ │ │ └─ llm.py             - OrchestratorAgent bridge
      │ │ │ │ └─ session.py         - AgentSession: STT/TTS/VAD wiring
      │ │ │
      │ │ │ └─ database/
      │ │ │ │ └─ crud.py
      │ │ │ │ └─ models.py
      │ │ │ │ └─ seed_data.py
      │ │ │
      │ │ │ └─ logging/
      │ │ │ │ └─ setup.py
      │ │ │
      │ │ │ └─ observability/
      │ │ │ │ └─ metrics.py
      │ │ │ │ └─ middleware.py
      │ │ │ │ └─ tracing.py
      │ │ │
      │ │ │ └─ intent/              - LLM-driven intent routing
      │ │ │ └─ language/            - language/locale handling
      │ │ │
      │ │ │ └─ slots/
      │ │ │ │ └─ extractor.py
      │ │ │ │ └─ models.py
```

Components — Detail

`orchestration/__init__.py` — `ConversationOrchestrator`

The core of TaskOrbit and the sole entry point for processing a user message, via `process_message()` (text/non-streaming) and `process_message_stream()` (streaming). One class handles, in order: manual-transfer short-circuit, end-call detection, scope refusal checks (#168), intent detection, agent selection (including turn-1 entry-agent locking and `allowed_handoffs` gating, #71), workflow dependency resolution (`workflow_rules.py`, #71), slot extraction, system prompt construction, the LLM call, and tool dispatch (including handoff target resolution via `tools/agent_transfer.py`, #212).

Correction: earlier drafts of this page described a multi-file `orchestration/` package — `models.py`, `task_loader.py`, `session.py`, `engine.py`, `agent_router.py`, `planner.py`, `handoff_manager.py`. None of these exist. The orchestration logic lives entirely in `__init__.py`.

`agents/__init__.py`

Holds `BaseAgent` (abstract `handle_message()` + `get_task_definitions()`), the concrete agents — `SalesAgent`, `TechnicalSupportAgent`, `GeneralInquiryAgent`, `AppointmentManagementAgent`, and others — and `AgentRegistry`, the single constructor used everywhere an agent instance is needed (`AgentRegistry.create()`, `AgentRegistry.create_by_name()`).

Correction: there is no `agents/llm_adapter.py`, `base_agent.py`, `greeting_agent.py`, or `task_agent.py`. `BaseAgent` and all concrete agents live together in `agents/__init__.py`.

`integrations/llm/factory.py`

`get_llm_client()` returns the concrete client for whichever of the **four** providers (`OpenAI`, `Google /Gemini`, `OpenRouter`, `Ollama`) is set on a given `AgentConfig.llm`. Each concrete client is lazy-imported and validated against its required API key/URL in `Settings`. `_guard_provider_model_match` (#99) rejects a model name that belongs to a different provider before it reaches the client.

Correction: there is no `integrations/llm_client.py`, and the provider set is not "OpenAI, hardcoded" — see [System-Architecture §3](#) for the full breakdown.

`tools/agent_transfer.py`

Owns `resolve_transfer_target()` (#212), which replaced a fragile regex-based target normalizer in the orchestrator. Given a raw target value from tool config (a UUID, a slug like `inquiry-agent`, or a display name), it resolves the real target agent reliably across both the text and voice paths.

`tools/generic_api.py`

`GenericApiTool` (#66) — a single config-driven adapter for external HTTP APIs (CRM lookups, webhooks, etc.), rather than one hand-written client per external service. Config carries the URL template, `{{env.X}}` / `{{args.Y}}` substitution with an env-var allowlist, and a dot-path response extractor so the LLM always sees a stable response shape. See [Generic External Tool Adapter](#) for the developer flow.

`livekit_agent/session.py` + `livekit_agent/llm.py`

`session.py` builds the LiveKit `AgentSession`, wiring Silero VAD, the selected STT/TTS providers (Deepgram and ElevenLabs are both dual-capable, selected per agent — #135), and `llm.py`'s `OrchestratorAgent`, which bridges each voice turn into `ConversationOrchestrator`.

`api/routes/`

`conversations.py` is the text-path entry point into `ConversationOrchestrator`. `livekit.py` issues scoped, short-lived LiveKit room tokens (`POST /v1/livekit/token`). `agent_configs.py` and `user_agents.py` manage saved agent configurations. `tts.py` exposes standalone text-to-speech.

Correction: there is no `api/routes/pipeline.py`, `session_routes.py`, `token_routes.py`, or `history_routes.py`.

Corrections From Earlier Draft

The following files/packages appeared in a previous version of this page and **do not exist** in the codebase. They described an early sprint plan that was restructured before or during implementation:

`orchestration/models.py`, `orchestration/task_loader.py`, `orchestration/session.py`, `orchestration/engine.py`, `orchestration/agent_router.py`, `orchestration/planner.py`, `orchestration/handoff_manager.py`, `agents/llm_adapter.py`, `agents/base_agent.py`, `agents/greeting_agent.py`, `agents/task_agent.py`, `application/` (`session_service.py`, `conversation_service.py`), `services/` (`language_service.py`, `logging_service.py`, `log_records_service.py`), `integrations/livekit_client.py`, `integrations/deepgram_client.py`, `integrations/llm_client.py`, `integrations/elevenlabs_client.py`, `database/repositories.py`, `database/log_records_repository.py`, `database/migrations/`, `config/settings.py`, `config/providers.json`, `api/routes/pipeline.py`, `api/routes/session_routes.py`, `api/routes/token_routes.py`, `api/routes/history_routes.py`.

Package Responsibilities

Package	Role
<code>agents/</code>	Domain agent implementations + registry
<code>orchestration/</code>	Single-class conversation orchestrator — the core pipeline
<code>api/routes/</code>	FastAPI route handlers (text conversations, LiveKit tokens, agent config CRUD, TTS)
<code>integrations/llm/</code>	Four-provider LLM client factory
<code>tools/</code>	Tool implementations dispatched by the orchestrator (data extraction, end call, agent transfer, generic external API)
<code>livekit_agent/</code>	Voice pipeline: <code>AgentSession</code> wiring + orchestrator bridge
<code>database/</code>	SQLAlchemy models, CRUD helpers, seed data
<code>logging/</code>	Structured logging setup
<code>observability/</code>	Metrics, middleware, tracing
<code>intent/</code>	LLM-driven intent routing
<code>language/</code>	Language/locale handling
<code>slots/</code>	Slot extraction for data-collection tasks

See also: [System Architecture · Architecture — Runtime](#)

Architecture Decisions

Architecture Decisions — Sprint 1

Sprint: AMOS SS2026 — Sprint 1
Issue: #12 — Basic runnable backend foundation
Status: ✔ Merged

What was built this sprint

The first runnable backend pipeline: **text in** → **task orchestration** → **stubbed LLM** → **text out**.

The full E2E pipeline runs without any external API keys or voice infrastructure. A single HTTP call drives a complete conversation turn through the orchestration engine.

Implemented files

File	Package	Responsibility
models.py	orchestration/	Pydantic v2 models mirroring agent_task.schema.json
task_loader.py	orchestration/	Load and validate a task JSON file from disk
session.py	orchestration/	In-memory session state per conversation
engine.py	orchestration/	Step runner — dispatches each workflow action
llm_adapter.py	agents/	Stubbed LLM adapter, OpenAI path ready to activate
pipeline.py	api/routes/	POST /pipeline/run — sprint 1 E2E entry point
sales.py	agents/	Sales domain agent
technical_support.py	agents/	Technical support domain agent
data_extraction.py	tools/	Stub — extract caller fields
end_call.py	tools/	Stub — terminate session
agent_transfer.py	tools/	Stub — handoff to another agent

Unit tests: 16 test cases across `test_task_loader` , `test_engine` , `test_llm_adapter` .

Decision 1 — Python as the backend language

Decision: Python 3.11 with FastAPI and Pydantic v2.

Rationale:

- The LiveKit Agents SDK has first-class Python support. Node.js support exists but is less mature for voice agent workloads.
- Pydantic v2 provides JSON schema validation that maps directly to our `agent_task.schema.json` contract with zero additional library code.
- The AI/ML ecosystem — LLM clients, STT/TTS SDKs — is predominantly Python-first.
- FastAPI gives us async support, automatic OpenAPI docs, and Pydantic integration out of the box.

Alternative considered: Node.js / TypeScript — rejected because LiveKit's Python SDK is the primary supported path for agent workers, and splitting orchestration logic across two runtimes adds unnecessary operational complexity.

Decision 2 — LLM adapter is stubbed this sprint

Decision: `llm_adapter.py` returns a deterministic placeholder response. No real API call is made.

Rationale: OpenAI API keys are not yet available. The stub allows the full pipeline to run end-to-end in tests and demos without any external dependency.

How to activate next sprint:

- Set the `OPENAI_API_KEY` environment variable
- In `llm_adapter.py`, replace `return self.stub_complete(...)` with `return self.call_openai(...)`

The real implementation is already written — it just needs uncommenting.

Decision 3 — Tools are stubbed this sprint

Decision: `data_extraction.py` , `end_call.py` , and `agent_transfer.py` log their invocation and return placeholders.

Rationale: Real tool execution requires the Meisterwerk API integration and session persistence — both future sprint work. Stubbing allows the engine to reach and log tool steps correctly, proving the orchestration logic is wired up, without blocking on external dependencies.

Conscious deviations from the architecture diagram

API shape: one route instead of three

Diagram: `session_routes.py` , `token_routes.py` , `history_routes.py`

Built: `pipeline.py` with `POST /pipeline/run`

Sessions as a REST resource require the database layer and LiveKit token infrastructure — both future sprint work. The single pipeline endpoint is sufficient for the first E2E text pipeline. It will be refactored into proper session routes when persistence is added.

Impact: The frontend `ApiClient` cannot yet use the final API shape. Expected — frontend integration is a later sprint.

No application service layer

Diagram: session_service.py , conversation_service.py
Built: Session management lives directly in pipeline.py

With a single API route and no persistence, a service layer adds indirection without benefit. When the LiveKit agent needs to trigger conversations from a separate entry point, the service layer will be extracted at that point.

Orchestration: one engine instead of router + planner + handoff

Diagram: agent_router.py , planner.py , handoff_manager.py
Built: engine.py handles all three responsibilities

Multi-agent routing and context handoffs are not needed for a single-task pipeline. The engine's _execute_step method is the natural split point when multi-agent support is introduced.

No agent base class hierarchy yet

Diagram: base_agent.py with greeting_agent and task_agent subclasses
Built: llm_adapter.py + domain agents without shared base class

The greeting vs task distinction is handled by workflow step action values in the engine. Separate agent classes with a base class make sense once agents have meaningfully different behaviour beyond their system prompt — that point arrives with multi-agent handoffs.

What remains for future sprints

Package	Sprint	Dependency
Real LLM integration	Sprint 2	OpenAI API key
LiveKit voice pipeline	Sprint 2	LiveKit credentials
STT + TTS adapters	Sprint 2	Deepgram + ElevenLabs keys
application/ service layer	Sprint 2	LiveKit agent needing shared entry point
config/ + providers.json	Sprint 2	Centralised settings
database/ + migrations	Sprint 2+	PostgreSQL persistence
services/ logging + language	Sprint 2+	Live pipeline running
Full REST session API	Sprint 2+	Database layer
Frontend integration	Sprint 3+	Final API shape

See also: [Tech Stack Decision · Architecture — Code Components](#)

Tech Stack Decision

Tech Stack Decision

This page documents the technology choices for the TaskOrbit backend and the reasoning behind them.

Backend

Technology	Version	Role
Python	3.11	Primary backend language
FastAPI	Latest	HTTP API framework
Pydantic v2	Latest	Data validation + schema enforcement
Poetry	Latest	Dependency management
structlog	Latest	Structured logging
pytest	Latest	Unit testing

Why Python over Node.js?

The decision came down to three concrete factors:

- LiveKit SDK maturity.** LiveKit's Python Agents SDK is the primary supported path for building voice agent workers. The Node.js SDK exists but is documented as less mature for agent workloads. Since LiveKit is load-bearing infrastructure for us, we follow the best-supported path.
- Pydantic + JSON schema.** Our entire orchestration logic is driven by `agent_task.schema.json`. Pydantic v2 validates JSON against Python models directly — no extra library, no glue code. The schema becomes runnable validation with one decorator.
- AI/ML ecosystem.** Every LLM client library (OpenAI, Anthropic, Google), every STT/TTS SDK, and every AI framework is Python-first. We will not fight the ecosystem.

Frontend

Technology	Version	Role
React	19	UI framework
Vite	5	Build tool + dev server
TypeScript	Latest	Type safety
Tailwind CSS	Latest	Styling
shadcn/ui	Latest	Component library
LiveKit Client SDK	Latest	WebRTC audio in browser

Infrastructure

Technology	Role
Docker + docker-compose	Local dev environment
PostgreSQL	Persistence (Sprint 2+)
Google Cloud Platform (GCP)	Production deployment target
LiveKit Cloud	Realtime audio transport
Alembic	Database migrations

External AI Providers

Role	Provider	Model	Status
LLM	OpenAI	gpt-4o-mini	(API key pending)
STT	Deepgram	nova-3	
TTS	ElevenLabs	eleven_multilingual_v2	

LLM provider note: The LLM provider is currently hardcoded to the OpenAI path in `llm_adapter.py`. The model itself is read from the task JSON's `pipeline.llm` block — so switching models requires only changing the task file, not the code. Switching providers (e.g. to Gemini) requires updating `llm_adapter.py` once a decision is made.

See also: [Getting Started](#)

Basic Agent Runtime

Basic Agent Runtime (#33)

Overview

The basic agent runtime implements the core pipeline for processing a user text prompt end-to-end:

Prompt → Intent Detection → Agent Selection → Tool Selection → System Prompt → LLM → Response

Entry point: `ConversationOrchestrator.process_message()` in `backend/src/taskorbit/orchestration/__init__.py`.

Pipeline Steps

Step	Component	Status
1. Detect intent	IntentRouter	Live — LLM-based classification with a confidence gate; see below
2. Select agent	AgentRegistry	Wired
3. Select active tool	<code>agent.get_task_definitions()</code>	Wired (first tool in scope)
4. Build system prompt	<code>_build_system_prompt()</code>	Wired
5. Call LLM	<code>_call_llm()</code>	Real providers wired in #18
6. Return response	ConversationResponse	Wired

Mocked Assumptions

Intent Detection

This section originally described the #33-era stub: `MockIntentDetector` always returned the `book_service_appointment` workflow regardless of prompt content.

That stub is no longer the live mechanism. `IntentRouter` (`backend/src/taskorbit/intent/__init__.py`) is wired into `ConversationOrchestrator.__init__` (`self._intent_router = IntentRouter()`) and performs LLM-based classification against `_KNOWN_INTENTS` with a confidence gate -- `requires_clarification=True` is set when confidence is below threshold or the LLM returns no matching intent. Its docstring is explicit: "Replaces `MockIntentDetector` in the orchestration pipeline."

`MockIntentDetector` still exists, but only as `IntentRouter` 's internal fallback (`self._fallback = MockIntentDetector()`) and for use in tests -- it is not the live routing path.

LLM Response

At the time of #33, `_call_llm` returned a hardcoded `[Mocked LLM] Received: "..."` string with a simulated `asyncio.sleep(0.5)` delay. Real `OpenAI/Gemini` providers were wired in #18 and are now active.

Structured Response

`ConversationResponse` always returns:

```python

`ConversationResponse( conversation_id: str # echoed from request reply: Message # assistant message from LLM  
selected_intent: str # e.g. "book_service_appointment"  
selected_agent: str # e.g. "Sales Bot" status: str # "success" or "error"  
error: str # populated on failure, empty on success )`

Error Handling

|                            |                                                            |
|----------------------------|------------------------------------------------------------|
| Scenario                   | Behaviour                                                  |
| No user message in request | status: error, descriptive error field                     |
| Whitespace-only message    | status: error, descriptive error field                     |
| LLM timeout                | status: error, friendly reply, error: "LLM call timed out" |
| Unexpected exception       | status: error, generic reply, error logged via structlog   |

Configuration

| Setting     | Env Var             | Default | Description                        |
|-------------|---------------------|---------|------------------------------------|
| LLM timeout | LLM_TIMEOUT_SECONDS | 10.0    | Seconds before LLM call is aborted |

Agent Routing

AgentRegistry.get\_agent() routes by keyword in agent\_config.id:

|                       | Keyword in ID            | Agent         |
|-----------------------|--------------------------|---------------|
|                       | support, technical, tech |               |
| TechnicalSupportAgent |                          | anything else |
| SalesAgent            |                          |               |

**Note:** SalesAgent.handle\_message() and TechnicalSupportAgent.handle\_message() currently delegate back to orchestrator.process\_message(). Real intent routing (IntentRouter) already replaced MockIntentDetector in the pipeline; full agent-owned handling of each step is separate, tracked follow-up work.

Tests

23 unit tests in backend/tests/test\_agent\_runtime.py and backend/tests/test\_orchestration.py covering:

- Happy path (success status, intent/agent fields populated)
- Agent registry routing
- Empty and whitespace-only message errors
- LLM timeout handling

Run with: cd backend  
poetry run pytest tests/test\_agent\_runtime.py tests/test\_orchestration.py -v

# Agent Routing Foundation

## Overview

This document describes the Agent Routing Foundation feature implemented for TaskOrbit. It replaces the original hard-coded keyword-based `MockIntentDetector` with a fully dynamic, LLM-driven intent routing pipeline that supports confidence gating, structured slot collection, tool dispatch, and multi-agent handoff.

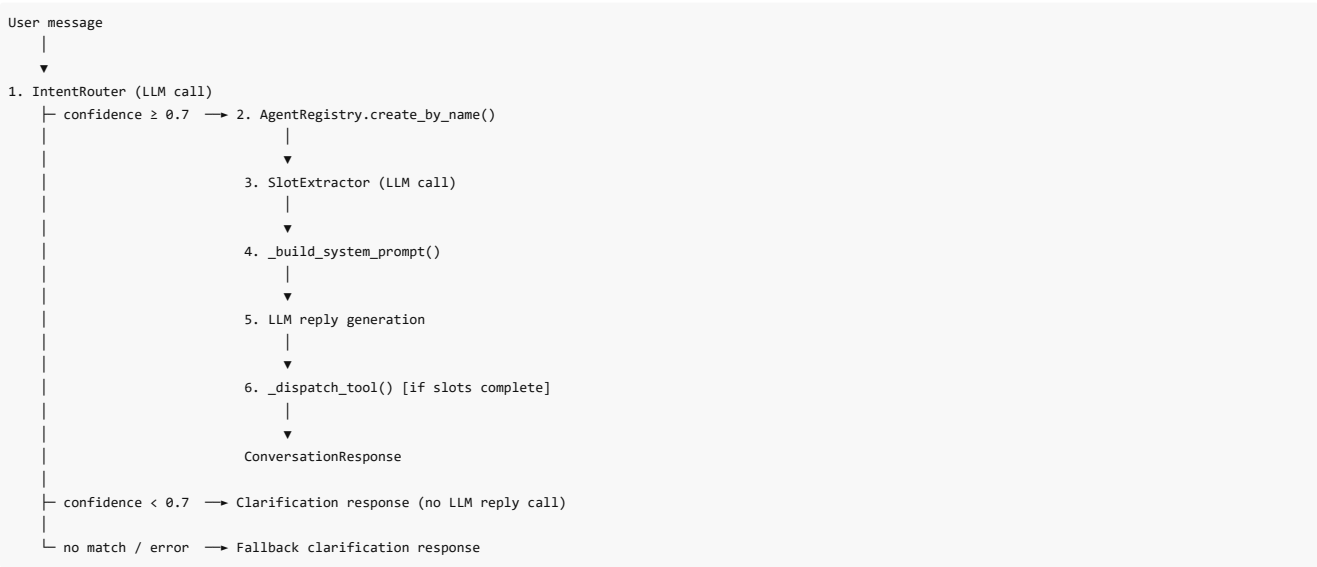
## Problem Statement

The original system had two hard-coded intents detected via keyword matching and an `AgentTransferTool` that raised `NotImplementedError`. This meant:

- Every message was routed to the same agent regardless of user intent
- No confidence scoring — ambiguous inputs were silently mis-routed
- No structured data collection pipeline
- No live agent handoff capability

## Architecture

### Pipeline per Message



### Key Components

| Component                | File                                    | Role                                                            |
|--------------------------|-----------------------------------------|-----------------------------------------------------------------|
| IntentRouter             | src/taskorbit/intent/__init__.py        | LLM-based intent classification with confidence gating          |
| MockIntentDetector       | src/taskorbit/intent/__init__.py        | Keyword fallback used when LLM classification fails             |
| AgentRegistry            | src/taskorbit/agents/__init__.py        | Factory that constructs agents by name or config keyword        |
| ConversationOrchestrator | src/taskorbit/orchestration/__init__.py | Main pipeline entry point                                       |
| SlotExtractor            | src/taskorbit/slots/extractor.py        | LLM-driven structured data extraction from conversation history |
| AgentTransferTool        | src/taskorbit/tools/agent_transfer.py   | Hands off conversation to a different agent                     |
| DataExtractionTool       | src/taskorbit/tools/data_extraction.py  | Emits collected slot data when a workflow completes             |
| EndCallTool              | src/taskorbit/tools/end_call.py         | Signals end of conversation                                     |
| seed_data.py             | src/taskorbit/database/seed_data.py     | Default agent templates inserted into the database              |

## Intent Detection

### How It Works

`IntentRouter.detect()` sends the current user message to the configured LLM with a system prompt that lists all known intents and their descriptions. The LLM responds with:

```
{ "intent": "<intent_name>", "confidence": 0.85 }
```

- If `confidence >= 0.7` → the matched intent is returned and the pipeline continues
- If `confidence < 0.7` → `requires_clarification=True` is set; the orchestrator short-circuits and returns a clarification question to the user
- If no intent matches or the LLM returns `null` → `_FALLBACK_RESULT` is returned with `requires_clarification=True`
- If the LLM call fails entirely (network/parse error) → falls back to keyword-based `MockIntentDetector` with `confidence=0.5`, which also triggers clarification since  $0.5 < 0.7$

### Confidence Threshold

```
CONFIDENCE_THRESHOLD = 0.7 # src/taskorbit/intent/__init__.py
```

### Clarification Message

"I want to make sure I help you with the right thing – could you tell me a bit more about what you're looking for?"

### Known Intents

Five intents are registered in `_KNOWN_INTENTS` :

| Intent                           | Agent                        | Required Inputs                                                      |
|----------------------------------|------------------------------|----------------------------------------------------------------------|
| book_service_appointment         | SalesAgent                   | caller_name, email_address, phone_number, preferred_date             |
| customer_dissatisfaction_inquiry | CustomerDissatisfactionAgent | caller_name, email_address, complaint_description, preferred_contact |
| technical_support_request        | TechnicalSupportAgent        | caller_name, issue_description                                       |
| general_inquiry                  | GeneralInquiryAgent          | (none)                                                               |
| appointment_management           | AppointmentManagementAgent   | caller_name, booking_reference, requested_action                     |

To add a new intent, add an `IntentResult` entry to `_KNOWN_INTENTS` in `src/taskorbit/intent/__init__.py` . `IntentRouter` picks it up automatically — no other code changes needed.

### Agents

#### Agent List

| Agent Class                  | agent_name               | Purpose                                     |
|------------------------------|--------------------------|---------------------------------------------|
| SalesAgent                   | sales                    | Lead qualification, appointment booking     |
| TechnicalSupportAgent        | technical_support        | Troubleshooting, issue diagnosis            |
| GeneralInquiryAgent          | general_inquiry          | FAQs, product/policy questions              |
| AppointmentManagementAgent   | appointment_management   | Reschedule, cancel, check existing bookings |
| CustomerDissatisfactionAgent | customer_dissatisfaction | Complaint capture, escalation               |

#### Adding a New Agent

1. Subclass `BaseAgent` in `src/taskorbit/agents/__init__.py`
2. Set `agent_name` and implement `handle_message()` + `get_task_definitions()`
3. Add a row to `AgentRegistry._REGISTRY` with matching keywords
4. Add a corresponding `IntentResult` to `_KNOWN_INTENTS` in `intent/__init__.py`
5. Add a seed template to `src/taskorbit/database/seed_data.py`
6. Re-run `poetry run python scripts/seed_defaults.py`

#### Agent Selection

Agents are selected by `agent_name` from the `IntentResult` — not by `config.id` . This means the LLM drives routing, not the static config passed in the request.

```
agent = AgentRegistry.create_by_name(intent.agent_name, request.agent_config, self)
```

### Slot Extraction

#### How It Works

After intent detection, `SlotExtractor` makes an LLM call over the full conversation history to extract the required fields defined in the intent's `required_inputs` .

- Fields explicitly stated by the user are extracted and validated
- Fields not mentioned are marked `missing`
- The system prompt is augmented with "Collected so far: ..." and "Still need from user: ..." so the LLM naturally asks for outstanding fields
- Tool dispatch is blocked until `slot_result.is_complete` is `True`

#### Validators

Built-in format validators prevent clearly malformed values from being stored:

| Type  | Validation Rule                                          |
|-------|----------------------------------------------------------|
| email | Must match <code>^[^@\\s]+@[^\\s]+\\.([^@\\s]+)\$</code> |
| phone | Must contain at least 7 digits                           |
| date  | Must match <code>YYYY-MM-DD</code>                       |

#### Defining Required Inputs

Each intent declares its required fields:

```
required_inputs=[
 {"name": "caller_name", "type": "string", "required": True},
 {"name": "email_address", "type": "email", "required": True},
 {"name": "preferred_date", "type": "date", "required": True},
 {"name": "service_location", "type": "location", "required": False},
]
```

Optional fields ( `required: False` ) are extracted if present but never block workflow completion.

### Tool Dispatch

Tools are dispatched automatically once all required slots are filled. The dispatch table in `ConversationOrchestrator._dispatch_tool()` :

| ToolType        | Implementation     | Effect                                  |
|-----------------|--------------------|-----------------------------------------|
| DATA_EXTRACTION | DataExtractionTool | Emits collected slot data               |
| AGENT_TRANSFER  | AgentTransferTool  | Transfers conversation to another agent |
| END_CALL        | EndCallTool        | Sets response status to ENDED           |

The dispatched tool and its result are returned in `ConversationResponse.tool_invoked` so the frontend can react (e.g. update the displayed agent label on transfer).

**AgentTransferTool**

Validates `target_agent_id` against all registered agent names, preserves the full conversation history, and returns:

```
{
 "transferred_to": "technical_support",
 "history_preserved": true,
 "message_count": 6
}
```

**ConversationResponse Fields**

| Field             | Type                  | Description                                     |
|-------------------|-----------------------|-------------------------------------------------|
| conversation_id   | str                   | Echoed from the request                         |
| reply             | Message               | Assistant message to send to the user           |
| status            | ConversationStatus    | success, clarification, ended, error            |
| selected_intent   | str                   | Name of the detected intent                     |
| selected_agent    | str                   | agent_name of the agent that handled the turn   |
| intent_confidence | float                 | Router confidence score (0.0–1.0)               |
| extracted_slots   | dict                  | Filled slot values (populated when is_complete) |
| missing_slots     | list[str]             | Slot names still needed from the user           |
| tool_invoked      | ToolDefinition   None | Tool that was dispatched this turn, if any      |
| error             | str                   | Error message when status == error              |

**Clarification & Fallback Behaviour**

| Scenario                              | Behaviour                                                                   |
|---------------------------------------|-----------------------------------------------------------------------------|
| LLM confidence < 0.7                  | Returns clarification question, no reply LLM call made                      |
| Unknown / null intent from LLM        | Returns clarification question                                              |
| LLM classification call fails         | Falls back to keyword matching with confidence=0.5 → triggers clarification |
| Missing required slots                | LLM is guided to ask the user; tool not dispatched                          |
| Malformed slot value (e.g. bad email) | Slot treated as missing; user asked to repeat                               |
| Empty / blank user message            | Returns status=error immediately                                            |

**Default Agent Templates**

Five agent templates are seeded into the database:

| ID                             | Name                           |
|--------------------------------|--------------------------------|
| sales-agent                    | Sales Agent                    |
| technical-support-agent        | Technical Support Agent        |
| general-inquiry-agent          | General Inquiry Agent          |
| appointment-management-agent   | Appointment Management Agent   |
| customer-dissatisfaction-agent | Customer Dissatisfaction Agent |

To seed or re-seed:

```
cd backend
poetry run python scripts/seed_defaults.py
```

The script is idempotent — safe to run any number of times.

**Testing**

**Running Tests**

```
cd backend
poetry run pytest tests/test_agent_runtime.py tests/test_agent_transfer.py -v
```

Test Coverage

| File                         | What It Covers                                                                        |
|------------------------------|---------------------------------------------------------------------------------------|
| tests/test_agent_runtime.py  | MockIntentDetector, AgentRegistry routing, orchestrator happy path and error handling |
| tests/test_agent_transfer.py | AgentTransferTool success, unknown agent ID, history preservation                     |

Integration Testing via Swagger

1. Start the backend: `docker compose up backend`
2. Open `http://localhost:8000/docs`
3. POST to `/v1/conversations/process` with a user message
4. Verify `selected_intent` , `selected_agent` , `intent_confidence` , and `status` in the response

Example — complaint routing:

```
{
 "conversation_id": "test-001",
 "agent_config": { "id": "sales-agent", "name": "Test", "persona": "Assistant", "greeting": "Hi" },
 "messages": [{ "role": "user", "content": "I am very unhappy with the service I received" }]
}
```

Expected: `selected_intent = "customer_dissatisfaction_inquiry"` , `selected_agent = "customer_dissatisfaction"` , `status = "success"` .

Known Limitations / Future Work

| Item                                                | Notes                                                                                                                                                    |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Two LLM calls per turn                              | One for intent classification, one for reply — doubles voice latency. Fix: cache intent after first turn and skip re-classification on subsequent turns. |
| Orchestrator is stateless                           | Active agent is not persisted between turns; intent is re-detected every message.                                                                        |
| LiveKit frontend continuity (Task 6)                | Frontend should read <code>tool_invoked.type == "agent_transfer"</code> and update the displayed agent label without disconnecting the room.             |
| <code>test_intent_router.py</code> missing (Task 7) | Unit tests for <code>IntentRouter</code> (high-confidence, low-confidence, unknown intent, parse failure) are not yet written.                           |

# Transitive Workflow Dependencies

TaskOrbit runs **multi-agent prerequisite chains** over **voice**. When agent A depends on B and B depends on C, the engine runs **C → B → A** at runtime — even when **A** only lists B in Agent Config.

**Note:** TaskOrbit is **voice-first**. The typed text input is legacy and will be **removed in upcoming sprints**. Manual testing uses **mic** → **Send** and the **Proceed** card. Backend automated tests still use text HTTP requests.

### Direct vs transitive

| Term                  | Meaning                                                     |
|-----------------------|-------------------------------------------------------------|
| Direct dependency     | Agent IDs in <b>Prerequisite Steps</b> for that agent       |
| Transitive dependency | Upstream agents inferred from saved configs in the database |

### Example configuration:

```
customer-entry → sales-qualification
sales-qualification → identity-verification
```

**Runtime order:** identity-verification → sales-qualification → customer-entry

The entry agent does **not** need to list every upstream step.

### Prerequisites

```
From repo root
docker compose up --build

LiveKit worker (required for voice)
cd backend
poetry run taskorbit-worker dev
```

Set in backend/.env :

- LIVEKIT\_URL
- LIVEKIT\_API\_KEY
- LIVEKIT\_API\_SECRET
- DEEPGRAM\_API\_KEY
- ELEVENLABS\_API\_KEY
- ELEVENLABS\_VOICE\_ID

Allow **microphone** in the browser. Use a **new session** for each test run.

### Example agents

Save in this order: **identity** → **sales** → **entry** (use **Save as new** in Agent Config).

#### 1. identity-verification (leaf)

| Field              | Value                                                                             |
|--------------------|-----------------------------------------------------------------------------------|
| Agent ID           | identity-verification                                                             |
| Display name       | Identity Verification                                                             |
| Instructions       | You verify the customer's identity. Always start your reply with exactly: VERIFY: |
| Greeting           | Identity check ready.                                                             |
| STT / LLM / TTS    | e.g. nova-3, gpt-4o-mini, any ElevenLabs voice                                    |
| Prerequisite Steps | (empty)                                                                           |
| Allowed Handoffs   | (empty)                                                                           |

#### 2. sales-qualification

| Field              | Value                                                                                       |
|--------------------|---------------------------------------------------------------------------------------------|
| Agent ID           | sales-qualification                                                                         |
| Display name       | Sales Qualification                                                                         |
| Instructions       | You qualify purchase intent and product needs. Always start your reply with exactly: SALES: |
| Greeting           | Sales qualification ready.                                                                  |
| Prerequisite Steps | identity-verification                                                                       |
| Allowed Handoffs   | (empty)                                                                                     |

Expected in saved JSON:

```
"workflow_dependencies": ["identity-verification"]
```

3. customer-entry (open in Chat)

| Field              | Value                                                                          |
|--------------------|--------------------------------------------------------------------------------|
| Agent ID           | customer-entry                                                                 |
| Display name       | Customer Entry                                                                 |
| Instructions       | You are the main reception agent. Always start your reply with exactly: ENTRY: |
| Greeting           | Welcome to customer support. How can I help you today?                         |
| Prerequisite Steps | sales-qualification only — do <b>not</b> add identity-verification             |
| Allowed Handoffs   | (empty)                                                                        |

Expected in saved JSON:

```
"workflow_dependencies": ["sales-qualification"]
```

Hard-refresh after saving if prerequisites are missing from the dropdown.

Voice test script

- 1. Agent Config → load customer-entry → Update
- 2. Chat → Start session — confirm Voice session active
- 3. Session title should stay Customer Entry
- 4. Every user turn: mic → speak → Send

| Turn | You do                               | Pass if                                             |
|------|--------------------------------------|-----------------------------------------------------|
| 1    | Say I want to buy something → Send   | Proceed for Identity Verification first (not Sales) |
| 2    | Click Proceed (or say proceed / yes) | Acknowledgment for identity step                    |
| 3    | Say continue → Send                  | Reply contains VERIFY:                              |
| 4    | Say continue → Send                  | Proceed for Sales Qualification                     |
| 5    | Click Proceed                        | Acknowledgment for sales step                       |
| 6    | Say continue → Send                  | Reply contains SALES: (not ENTRY:)                  |
| 7    | Say continue → Send                  | Reply contains ENTRY:; no more Proceed cards        |

Voice confirm words: yes , proceed , sure , ok , go ahead

Extra text after each prefix is OK — only order and prefix matter.

Minimal test IDs (QA)

Same graph with short names for internal QA:

| Realistic ID          | QA ID   | Prefix  |
|-----------------------|---------|---------|
| identity-verification | agent-c | STEP-C: |
| sales-qualification   | agent-b | STEP-B: |
| customer-entry        | agent-a | STEP-A: |

Workflow Proceed card

The Proceed / Cancel card shows a colored left border on workflow steps. That is intentional UI styling, not an error.

Clicking Proceed syncs workflow state to the LiveKit worker via the workflow\_state data channel.

UI Proceed sync test (optional)

Repeat the voice script once using UI Proceed only (do not say yes into the mic) for the Identity and Sales confirmation steps. Voice continue turns should still produce the correct prefixes in order.

Automated tests

```
cd backend
poetry run pytest tests/test_workflow_engine.py -q
```

Troubleshooting

| Symptom                          | Check                                                        |
|----------------------------------|--------------------------------------------------------------|
| No voice reply                   | Worker running; LiveKit / Deepgram / ElevenLabs keys in .env |
| Prerequisite missing in dropdown | Save upstream agents first; hard refresh                     |
| Sales Proceed before Identity    | sales-qualification must list identity-verification          |

|                                 |                                                           |
|---------------------------------|-----------------------------------------------------------|
| Wrong persona after Proceed     | Restart backend + worker                                  |
| Title shows "General Inquiry"   | Use latest build; title should stay <b>Customer Entry</b> |
| UI Proceed but wrong voice step | Worker workflow_state sync; hard refresh                  |
| No Proceed card                 | Entry agent has prerequisites; STT/LLM/TTS models filled  |

---

---

# Audio Input via LiveKit — STT

## Overview

TaskOrbit uses **Deepgram** for speech-to-text, wired into a **livekit-agents** `AgentSession` . Audio flows from the browser microphone through a LiveKit room, gets transcribed, and lands in the orchestrator as a plain-text message. The pipeline is push-to-talk: the backend only produces a reply when the frontend explicitly commits the turn — either the user presses **Send**, or the frontend's own silence detection auto-commits after ~1.5 seconds of quiet. A server-side Silero VAD backstop nudges the frontend to commit early if background noise is preventing its own detection from firing (see "Two Modes" below).

## Full Pipeline



## Turn/Message Commit: Two Modes

### 1. Manual Send (Push-to-Talk)

The user clicks the **Send** button in the UI. The frontend publishes a `{ "type": "commit_turn" }` data packet over the LiveKit data channel. The worker receives it and triggers `session.generate_reply()` .

```
User clicks Send
→ frontend: localParticipant.publishData({ "type": "commit_turn" })
→ worker: data_received event fires
→ agent.request_reply() + session.generate_reply()
```

### 2. Auto-Send on Silence (~1.5-second pause)

When the user stops speaking, the frontend automatically commits the turn — no button press needed. This runs on amplitude, not the backend's Silero VAD:

```
User stops speaking
→ frontend useSilenceDetection.ts polls mic amplitude (Web Audio API, every 100 ms)
→ amplitude stays below SILENCE_THRESHOLD (26 of 255) for SILENCE_DURATION_MS (1500 ms)
→ publishData({ "type": "commit_turn" })
→ same path as manual Send
```

**Backstop:** the backend's Silero VAD independently detects end-of-speech server-side ( `turn_handling.turn_detection = "vad"` ). On that transition it publishes a `force_commit` nudge over the `taskorbit.force_commit` LiveKit data topic — this rescues the case where ambient noise holds the browser's amplitude analyser above its threshold so the frontend's own detection never fires. The frontend ignores the nudge if it has already committed.

The `1500 ms` figure and the `26 / 30` thresholds are hardcoded constants in `frontend/src/hooks/useSilenceDetection.ts` (tuned for #153, down from 2000 ms / up from 20). They are **not** currently read from any per-agent config field — see Known Limitations below.

Both paths converge at the same `commit_turn` handler in the worker — the backend has no distinction between the two trigger sources.

## Backend

### STT Configuration

File: `backend/src/taskorbit/config.py`

| Setting           | Default           | Description                        |
|-------------------|-------------------|------------------------------------|
| DEEPGRAM_API_KEY  | <i>(required)</i> | Deepgram API key                   |
| DEEPGRAM_MODEL    | nova-3            | Most accurate Deepgram model       |
| DEEPGRAM_LANGUAGE | multi             | Auto-detect language per utterance |

`.env` example:

```
DEEPGRAM_API_KEY=dg...
DEEPGRAM_MODEL=nova-3
DEEPGRAM_LANGUAGE=multi
```

## AgentSession Wiring

File: backend/src/taskorbit/livekit\_agent/session.py

```
AgentSession(
 vad=silero.VAD.load(
 activation_threshold=0.7,
 deactivation_threshold=0.6, # raised 0.45→0.6 for #153 (ambient noise)
 min_speech_duration=0.2,
 min_silence_duration=1.5, # held at 1.5s by the #102 regression guard
 prefix_padding_duration=0.4,
),
 stt=deepgram.STT(
 api_key=cfg.deepgram_api_key,
 model=cfg.deepgram_model, # "nova-3"
 language=cfg.deepgram_language, # "multi"
),
 llm=openai.LLM(...), # placeholder – overridden by llm_node
 tts=elevenlabs.TTS(...),
 turn_handling={
 "turn_detection": "vad",
 "preemptive_generation": {"enabled": False},
 },
)
```

### Key decisions:

- **turn\_detection: "vad"** — Silero detects end-of-speech server-side and the framework commits its own turn boundary; this is more noise-robust than the browser's amplitude check alone (#153). It powers the `force_commit` backstop described above, but it does **not** by itself trigger a reply.
- **Reply generation stays explicitly gated regardless of turn\_detection** . `OrchestratorAgent.llm_node()` only produces output when `request_reply()` has previously been called (i.e. a `commit_turn` data message was received); the framework's own automatic VAD end-of-turn call into `llm_node` is detected and silently dropped so it can't double-fire a reply.
- **preemptive\_generation: {"enabled": False}** — the framework's default is `enabled=True` ; disabling it stops the framework from invoking `llm_node` early on a partial/pre-flush transcript, so the worker's commit-driven reply (which reads the live finalized transcript) is the single reply path.
- The **OpenAI LLM** is a placeholder — `generate_reply()` requires an LLM to be registered but `OrchestratorAgent.llm_node()` fully overrides it. The OpenAI model is never called or billed.

## Push-to-Talk Guard

File: backend/src/taskorbit/livekit\_agent/llm.py

Even with `turn_detection: "vad"` , livekit-agents can call `llm_node` preemptively on its own end-of-turn detection. A flag gates it:

```
class OrchestratorAgent(Agent):
 def __init__(self, ...):
 self._reply_requested: bool = False

 def request_reply(self) -> None:
 self._reply_requested = True # called before generate_reply()

 async def llm_node(self, chat_ctx, tools, model_settings):
 if not self._reply_requested:
 return # preemptive call – silently drop
 self._reply_requested = False
 # ... process and yield reply
```

## Worker Entrypoint

File: backend/src/taskorbit/worker.py

The worker handles `commit_turn` regardless of whether it came from a manual Send or the ~1.5-second auto-send — the packet is identical:

```
@ctx.room.on("data_received")
def _on_data(packet: rtc.DataPacket) -> None:
 msg = json.loads(packet.data.decode("utf-8"))
 if msg.get("type") == "commit_turn":
 agent.request_reply()
 session.generate_reply()
```

## Chat Context – Orchestrator

File: backend/src/taskorbit/orchestration/\_\_init\_\_.py

Uses only the **current turn's user messages** — everything after the last assistant message. This resets the buffer after each commit, whether manual or auto:

```
current_turn: list[Message] = []
for m in reversed(request.messages):
 if m.role == MessageRole.ASSISTANT:
 break
```

```
if m.role == MessageRole.USER:
 current_turn.insert(0, m)
```

**Deepgram overlap deduplication:** Deepgram sometimes sends the final word(s) as a trailing segment that duplicates the end of the full result. The join step skips any message already appearing as a suffix:

```
combined = ""
for m in current_turn:
 content = m.content.strip()
 if not combined:
 combined = content
 elif not combined.endswith(content):
 combined = combined + " " + content
```

Frontend

Transcription Hook

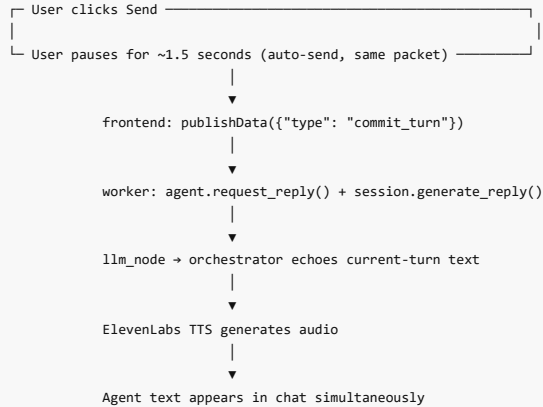
File: frontend/src/hooks/useAgentTranscription.ts

Subscribes to the `lk.transcription` LiveKit topic. Uses `TextStreamReader` async iteration — each yield is the cumulative text so far, giving word-by-word updates:

```
for await (const chunk of reader) {
 text = chunk;
 onSegment({ id: segmentId, role, text, isFinal: false });
}
if (text) onSegment({ id: segmentId, role, text, isFinal: true });
```

State Flow (Per Turn)

User taps mic → mic published → Silero VAD gates audio → Deepgram transcribes  
→ interim STT segments update user bubble in real time



Critical Files

| File                                            | Role                                                                                                                 |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| backend/src/taskorbit/config.py                 | Deepgram env vars                                                                                                    |
| backend/src/taskorbit/livekit_agent/session.py  | STT + VAD wiring, turn_detection: "vad"                                                                              |
| backend/src/taskorbit/livekit_agent/llm.py      | Push-to-talk guard (request_reply/_reply_requested), ChatContext → Message conversion                                |
| backend/src/taskorbit/worker.py                 | commit_turn handler (shared by manual + auto-send); publishes force_commit backstop on server-side VAD end-of-speech |
| backend/src/taskorbit/orchestration/__init__.py | Current-turn extraction, overlap deduplication                                                                       |
| frontend/src/hooks/useAgentTranscription.ts     | LiveKit transcription stream subscription                                                                            |
| frontend/src/hooks/useSilenceDetection.ts       | Amplitude-based silence detection driving the ~1.5 s auto-send                                                       |
| frontend/src/components/ConversationalChat.tsx  | Word accumulator; wires up auto-send via useSilenceDetection                                                         |
| frontend/src/types/agentConfig.ts               | vad.silence_threshold_ms field (unwired placeholder — see Known Limitations)                                         |

Verification Steps

1. Start the stack: `docker compose up --build`
2. Open `http://localhost:5173` → click **Start Session**
3. **Manual send:** speak a sentence, click Send → echo appears word-by-word with blinking cursor
4. **Auto-send:** speak a sentence, stop talking — after ~1.5 seconds the turn commits automatically without clicking Send
5. Worker logs should show `[STT] <your sentence>` exactly once per committed turn

Known Limitations

| Gap                            | Notes                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Language detection per-agent   | deepgram_language is global; no per-session override. language/__init__.py raises NotImplementedError.                                                                                                                                                                                                                                                                                                                                                           |
| Real LLM                       | Orchestrator currently echoes input. LLM wiring is a future sprint.                                                                                                                                                                                                                                                                                                                                                                                              |
| silence_threshold_ms not wired | This field only exists on the frontend's wider, forward-looking AgentConfig.vad type (frontend/src/types/agentConfig.ts) — it is not part of the backend's strict orchestrator AgentConfig (see <a href="#">Agent Task Schema</a> ) and is never read at runtime. The real ~1.5 s timing is two independent hardcoded values: SILENCE_DURATION_MS = 1500 in useSilenceDetection.ts (frontend) and min_silence_duration = 1.5 in the backend's Silero VAD config. |

# Audio Output via LiveKit — TTS

## Overview

TaskOrbit uses **ElevenLabs** to convert the orchestrator's text responses into speech. There are now **two TTS paths**:

1. **Voice path** — `AgentSession` (livekit-agents) synthesises audio via `elevenlabs.TTS` and publishes it as a LiveKit audio track, played by `RoomAudioRenderer` in the browser.
2. **Text path** — when the user types rather than speaks, the frontend calls `POST /v1/tts/synthesize` (a FastAPI route that proxies ElevenLabs directly) and plays the returned MP3 via `HTMLAudioElement`.

[Voice path]  
OrchestratorAgent reply (text) → AgentSession → elevenlabs.TTS plugin → LiveKit audio track → RoomAudioRenderer → browser speakers

[Text path]  
Assistant text reply → POST /v1/tts/synthesize → ElevenLabs REST API → MP3 blob → HTMLAudioElement → browser speakers

Provider configuration lives in `backend/src/taskorbit/config.py`. All API keys are injected via environment variables.

## Implementation Status

| Component                                     | File                                           | Status |
|-----------------------------------------------|------------------------------------------------|--------|
| TTS wiring (AgentSession factory)             | backend/src/taskorbit/livekit_agent/session.py | Done   |
| Orchestrator → LLM bridge (OrchestratorAgent) | backend/src/taskorbit/livekit_agent/llm.py     | Done   |
| TTS config (voice ID, model, API key)         | backend/src/taskorbit/config.py                | Done   |
| Text-path TTS REST endpoint                   | backend/src/taskorbit/api/routes/tts.py        | Done   |
| Frontend voice audio (RoomAudioRenderer)      | frontend/src/components/ConversationalChat.tsx | Done   |
| Frontend text-path TTS client                 | frontend/src/lib/ttsApi.ts                     | Done   |
| Multilingual TTS (language detection → voice) | backend/src/taskorbit/language/__init__.py     | Stub   |
| TTS unit tests                                | backend/tests/test_livekit_agent_tts.py        | Done   |

## Architecture Change from Sprint 2

The old `VoicePipelineAgent` + `_run_tts()` pattern (in-repo worker) has been replaced:

| Old                                                               | New                                                                         |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>VoicePipelineAgent</code>                                   | <code>AgentSession</code> (newer livekit-agents API)                        |
| <code>_run_tts()</code> in <code>livekit_agent/__init__.py</code> | <code>build_agent_session()</code> in <code>livekit_agent/session.py</code> |
| In-repo taskorbit-worker CLI runs the agent                       | <b>LiveKit Cloud Agents</b> dispatch — no in-repo worker process            |
| Single TTS path                                                   | Two paths: voice (LiveKit) and text ( <code>/v1/tts/synthesize</code> )     |

`livekit_agent/__init__.py` is now a thin re-export module; the implementation lives in `session.py` and `llm.py`.

## How TTS Works

### Voice Path — AgentSession (LiveKit)

`build_agent_session()` in `livekit_agent/session.py` constructs an `AgentSession`. TTS defaults to ElevenLabs but is selected per-agent (see "Per-Agent Provider Selection" below). LiveKit Cloud Agents dispatch the session; there is no in-repo worker process.

```
backend/src/taskorbit/livekit_agent/session.py
from livekit.agents import AgentSession
from livekit.plugins import deepgram, elevenlabs, openai, silero

def build_agent_session(*, settings=None, agent_config=None) -> AgentSession:
 cfg = settings or get_settings()
 stt = _build_stt(cfg, agent_config) # Deepgram or ElevenLabs, per agent_config.stt.provider
 tts = _build_tts(cfg, agent_config) # Deepgram or ElevenLabs, per agent_config.tts.provider
 return AgentSession(
 vad=silero.VAD.load(
 activation_threshold=0.7,
 deactivation_threshold=0.6,
 min_speech_duration=0.2,
 min_silence_duration=1.5,
 prefix_padding_duration=0.4,
),
 stt=stt,
 llm=openai.LLM(# placeholder — overridden by OrchestratorAgent.llm_node
 model=cfg.openai_model,
 api_key=cfg.openai_api_key or "sk-placeholder-not-used",
),
 tts=tts,
 turn_handling={
 "turn_detection": "vad", # server-side Silero end-of-turn (#153)
 "preemptive_generation": {"enabled": False},
 },
)
```

```
 },
)
}
```

The `OrchestratorAgent` in `livekit_agent/llm.py` overrides `llm_node()` to route inference through `ConversationOrchestrator` instead of the OpenAI plugin. It guards against preemptive generation calls via a `request_reply()` flag — the session only produces a reply when `generate_reply()` is called explicitly from the data channel handler.

#### Per-Agent Provider Selection (#135)

STT and TTS provider selection is no longer hardcoded to Deepgram/ElevenLabs — `build_agent_session()` calls `_build_stt(cfg, agent_config)` and `_build_tts(cfg, agent_config)`, which read `agent_config.stt.provider` / `agent_config.tts.provider` (`STTProvider` / `TTSProvider` — `"deepgram"` | `"elevenlabs"` for both stages independently) and construct the matching plugin. When `agent_config` is absent or a field is empty, the historical env defaults apply (Deepgram STT, ElevenLabs TTS).

`_build_tts` includes a Deepgram branch alongside the ElevenLabs one:

```
backend/src/taskorbit/livekit_agent/session.py
def _build_tts(cfg, agent_config):
 tts_cfg = agent_config.tts if agent_config is not None else None

 if tts_cfg is not None and tts_cfg.provider == TTSProvider.DEEPGRAM:
 # Aura voices are encoded in the model name; voice_id is unused here.
 model = _resolve_model(..., fallback="aura-2-andromeda-en")
 return deepgram.TTS(
 api_key=cfg.deepgram_api_key,
 model=model,
 encoding="linear16", # pinned explicitly so a plugin upgrade
 # cannot silently change audio characteristics
)
 # ... else falls through to the ElevenLabs branch shown above
```

Symmetrically, `_build_stt` has an ElevenLabs branch (`elevenlabs.STT`, default model `scribe_v2_realtime`) alongside the Deepgram one. Provider-selection source (config vs. env fallback) is logged distinctly (`stt_selected` / `tts_selected`, `source="config"` vs the no-config path) so a provider-switching failure in production is visible rather than silent.

#### Text Path — REST Endpoint

When the user sends a typed message, the frontend calls `POST /v1/tts/synthesize`. The FastAPI route proxies directly to the ElevenLabs REST API and returns MP3 audio:

```
backend/src/taskorbit/api/routes/tts.py
@router.post("/synthesize")
async def synthesize_speech(request: TTSSynthesizeRequest) -> Response:
 # Calls https://api.elevenlabs.io/v1/text-to-speech/{voice_id}
 # voice_settings: stability=0.75, similarity_boost=0.75, use_speaker_boost=True
 # Returns: audio/mpeg (MP3)
```

Request body fields (all optional except `text`):

- `text` — string, 1–5000 chars
- `voice_id` — overrides `ELEVENLABS_VOICE_ID` env var
- `model_id` — overrides `ELEVENLABS_MODEL` env var

#### Frontend

`ConversationalChat.tsx` uses both TTS paths:

```
import { RoomAudioRenderer } from "@livekit/components-react";
import { playSynthesizedSpeech } from "@lib/ttsApi";

// Voice path: RoomAudioRenderer auto-plays every remote audio track from the LiveKit room.
// Mounted inside <LiveKitRoom> — no additional code needed.
<RoomAudioRenderer />

// Text path: called after receiving a typed assistant reply
await playSynthesizedSpeech(assistantText, { signal: abortRef.current?.signal });
```

`playSynthesizedSpeech` (in `frontend/src/lib/ttsApi.ts`) calls `/api/v1/tts/synthesize`, creates a blob URL, plays it via `HTMLAudioElement`, then revokes the URL.

#### ElevenLabs Voice Options

Set `ELEVENLABS_VOICE_ID` in `backend/.env` to change the agent's voice. The current default is **Rachel**.

| Voice Name                | Voice ID              | Gender | Description                         |
|---------------------------|-----------------------|--------|-------------------------------------|
| Rachel ( <i>default</i> ) | 21m00Tcm4T1vDq8IkWAM  | Female | Calm, professional American English |
| Domi                      | AZnz1k1XvdvUeBnXm11d  | Female | Confident, expressive               |
| Bella                     | EXAVITQu4vr4xnSDxMaL  | Female | Warm, soft-spoken                   |
| Antoni                    | ErXwobaYiN019PkySvjV  | Male   | Well-rounded, natural               |
| Elli                      | MF3mGyEYc17XyWbV9V60  | Female | Conversational, young               |
| Josh                      | TxGEqnHmrFWFTfGw9XjX  | Male   | Deep, calm                          |
| Arnold                    | VR6AewLTigNG4xSOukaG  | Male   | Crisp, authoritative                |
| Adam                      | pNIznz6obpgDQ6cFmaJg8 | Male   | Deep, narrative                     |
| Sam                       | yoZ06aMxZjJ28mfd3POQ  | Male   | Raspy, strong                       |

To use a cloned or custom voice, paste its ID from your ElevenLabs dashboard directly into `ELEVENLABS_VOICE_ID`.

**Config location:** `backend/src/taskorbit/config.py`

```
elevenlabs_voice_id: str = "21m00Tcm4TlvDq8ikWAM" # override via ELEVENLABS_VOICE_ID env var
elevenlabs_model: str = "eleven_multilingual_v2"
```

### ElevenLabs Model

The project uses `eleven_multilingual_v2`, which supports 29 languages out of the box. No voice switching is required for multilingual responses — the same voice ID works across all supported languages.

Supported languages include: English, German, Spanish, French, Italian, Portuguese, Polish, Hindi, Arabic, and more.

### Multilingual TTS (Planned — Step 3)

`LanguageDetector` in `backend/src/taskorbit/language/__init__.py` is a stub. The scope has been narrowed to **English and German** (initial sprint):

```
class LanguageDetector:
 SUPPORTED: ClassVar[set[str]] = {"en", "de"}
 FALLBACK: ClassVar[str] = "en"

 def detect(self, text: str) -> str:
 raise NotImplementedError # ← still a stub
```

A companion `get_stt_language_code()` helper (also a stub) will map an `AgentConfig.stt_language` value to Deepgram's `language` parameter. Deepgram already defaults to `"multi"` (automatic multilingual mode) via `DEEPGRAM_LANGUAGE`.

ElevenLabs handles multilingual output automatically via `eleven_multilingual_v2` once language detection is wired in — no voice switching required.

### Required Environment Variables

```
ELEVENLABS_API_KEY=...
ELEVENLABS_VOICE_ID=21m00Tcm4TlvDq8ikWAM # optional — defaults to Rachel
ELEVENLABS_MODEL=eleven_multilingual_v2 # optional — defaults as shown

LiveKit Cloud Agent dispatch
LIVEKIT_URL=wss://...
LIVEKIT_API_KEY=...
LIVEKIT_API_SECRET=...
LIVEKIT_AGENT_DISPATCH_NAME=CA... # optional — LiveKit Cloud agent ID
```

See `backend/.env.example` for the full list.

### Critical Files

| File                                                         | Purpose                                                                                       |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>backend/src/taskorbit/livekit_agent/session.py</code>  | <code>build_agent_session()</code> — constructs <code>AgentSession</code> with ElevenLabs TTS |
| <code>backend/src/taskorbit/livekit_agent/llm.py</code>      | <code>OrchestratorAgent</code> — bridges orchestrator replies into the voice pipeline         |
| <code>backend/src/taskorbit/livekit_agent/__init__.py</code> | Re-export module (not the implementation)                                                     |
| <code>backend/src/taskorbit/api/routes/tts.py</code>         | <code>POST /v1/tts/synthesize</code> — text-path TTS REST endpoint                            |
| <code>backend/src/taskorbit/config.py</code>                 | ElevenLabs voice ID, model, API key, and LiveKit dispatch settings                            |
| <code>backend/src/taskorbit/language/__init__.py</code>      | Language detection stub (feeds multilingual TTS — Step 3)                                     |
| <code>frontend/src/components/ConversationalChat.tsx</code>  | <code>RoomAudioRenderer</code> (VOICE) + <code>playSynthesizedSpeech</code> (text)            |
| <code>frontend/src/lib/ttsApi.ts</code>                      | HTTP client for <code>POST /v1/tts/synthesize</code>                                          |
| <code>backend/tests/test_livekit_agent_tts.py</code>         | TTS unit tests (assert <code>build_agent_session</code> wires ElevenLabs correctly)           |

### Verification

#### Voice path

- Deploy the LiveKit Cloud Agent (wired via `LIVEKIT_AGENT_DISPATCH_NAME`)
- Start the backend: `poetry run taskorbit-api`
- Start the frontend: `npm run dev`
- Open browser → click "Start Call"
- Speak into mic → agent replies → audio plays through browser speakers via `RoomAudioRenderer`

#### Text path

- Start the backend: `poetry run taskorbit-api`
- Start the frontend: `npm run dev`
- Type a message in the chat input → agent replies → `playSynthesizedSpeech` plays the response

### Tests

```
TTS unit tests
poetry run pytest backend/tests/test_livekit_agent_tts.py
```

# Change voice: set ELEVENLABS\_VOICE\_ID in backend/.env and restart the backend

**Known Limitations**

| Gap                                     | Notes                                                                                                     |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Blinking cursor with Type Writer effect | The agent message should be returned in a typo text format instead of whole chunk being returned at once. |

# Voice Interruption for Audio Chat

## User Story

As a user, I want the voice agent to stop speaking immediately when I start talking, so that I can provide new information or corrections without having to wait for the agent to finish its response.

## Overview

The barge-in feature allows a user to interrupt the agent mid-speech. When sustained voice activity is detected from the user while the agent is playing audio, the agent's TTS output is cut immediately, any ongoing LLM generation or TTS tasks are cancelled, and the system transitions into a new recording turn.

The implementation is split across two layers:

- Frontend** — detects voice activity using the Web Audio API and fires an explicit interrupt signal.
- Backend** — LiveKit AgentSession is configured to honour interruptions and cancel in-flight TTS/LLM work.

## Implementation

### Frontend

`frontend/src/hooks/useVoiceActivityMonitor.ts`

A dedicated hook that opens a separate, unpublished `getUserMedia` stream purely for amplitude analysis. It does not publish audio to LiveKit — no network cost, no track visible to the agent. `echoCancellation: true` filters the agent's own TTS audio from the speakers, preventing the agent's voice from triggering a false self-interruption.

Detection logic:

| Parameter                      | Value                  | Rationale                                                                                                            |
|--------------------------------|------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>SPEECH_THRESHOLD</code>  | 55 (0–255)             | Rejects typical ambient noise (10–25) and brief background sounds (30–45). Clear speech reads ~60–120.               |
| <code>SPEECH_SUSTAIN_MS</code> | 500 ms                 | Rejects coughs, clicks, and short background bursts (<300 ms) while still feeling responsive for intentional speech. |
| <code>POLL_MS</code>           | 50 ms                  | Gives ≤350 ms total detection latency (sustain + 1 poll).                                                            |
| Frequency bins                 | 2–40 (~300 Hz – 3 kHz) | Covers speech fundamentals and formants; ignores low rumble and ultrasonic noise.                                    |

When amplitude exceeds the threshold for the full sustain window, `onSpeech()` fires exactly once. The hook then tears down the stream — it will not fire again until it is re-activated.

The hook is active only when `status === "speaking" && !greetingInProgress`. It is explicitly disabled during the agent's opening greeting to prevent ambient noise from cutting it off.

`frontend/src/components/chat/InCallControls.tsx` — `handleInterruptAndSpeak()`

Called by `useVoiceActivityMonitor` when speech is detected:

- `mic.sendInterrupt()` — fires a LiveKit data message telling the backend to stop TTS
- `mic.enable()` — publishes the mic track so recording begins immediately
- `onPhase("recording")` — updates the call state machine

`sendInterrupt()` is not awaited before `enable()` — both fire concurrently to minimise the gap between the agent going silent and the user's audio being captured.

### Mic Pre-Warm

To avoid losing the user's first words, the mic is pre-warmed as soon as the agent enters the `speaking` phase (when continuous mode is active). The LiveKit track is published and Deepgram begins receiving audio before the user decides to interrupt. This eliminates the 300–500 ms OS hardware cold-start latency that would otherwise swallow the first syllable.

```
useEffect(() => {
 if (!continuousMode) return;
 if (greetingInProgress) return;
 if (status !== "speaking") return;
 void mic.enable();
}, [status, continuousMode, greetingInProgress, mic]);
```

### Backend

`backend/src/taskorbit/livekit_agent/session.py` — `AgentSession` configuration

**Current state:** `allow_interruptions` and `min_interruption_duration` are **not** passed as explicit `AgentSession` kwargs — both are left at the LiveKit 1.6.0 framework defaults:

```
AgentSession(
 ...
 # No explicit allow_interruptions / min_interruption_duration here.
 # Framework defaults apply: min_duration=0.5s, resume_false_interruption=True.
 ...
)
```

| Setting                                | Effective value         | Effect                                                                                                           |
|----------------------------------------|-------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>allow_interruptions</code>       | True (LiveKit default)  | Enables livekit-agents to stop TTS and cancel in-flight LLM/synthesis tasks when an interruption signal arrives. |
| <code>min_interruption_duration</code> | 0.5 s (LiveKit default) | Ignores interruption signals shorter than 500 ms.                                                                |

A more aggressive override ( `min_duration=0.2` , `resume_false_interruption=False` ) was tried for snappier barge-in, but it tripped on ambient noise/echo mid-reply and cancelled the agent's own answers — so it was **reverted**. Tuning barge-in more aggressively needs the echo path (headphones / proper AEC) sorted first.

### Silero VAD — Noise Filtering

The VAD model running in the backend provides a second independent noise gate before any speech is forwarded to Deepgram STT or triggers agent logic.

```
silero.VAD.load(
 activation_threshold=0.7, # High confidence required to declare speech started
 deactivation_threshold=0.6, # Raised 0.45→0.6 (#153): steady ambient noise (e.g. a
 # fan) parked speech probability in the hysteresis band,
 # holding the turn open so end-of-speech never fired
 min_speech_duration=0.2, # Ignore sub-200 ms bursts (clicks, pops)
 min_silence_duration=1.5, # Held at 1.5s by the #102 regression guard – shorter
 # risks a mid-sentence pause splitting one utterance
 # into multiple transcript bubbles
 prefix_padding_duration=0.4, # Prepend 400 ms of audio to avoid cutting off word starts
)
```

Acceptance Criteria — Mapping

| Criterion                                               | How it is met                                                                                                                                                                                    |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Detect user voice activity while agent is playing audio | <code>useVoiceActivityMonitor</code> actively polls mic amplitude via Web Audio API during <code>status === "speaking"</code>                                                                    |
| Interrupt agent audio with minimal delay                | Frontend fires <code>sendInterrupt()</code> concurrently with <code>mic.enable()</code> ; mic pre-warm eliminates hardware cold-start latency                                                    |
| Cancel ongoing LLM / TTS tasks                          | <code>allow_interruptions=True</code> in <code>AgentSession</code> — <code>livekit-agents</code> cancels in-flight synthesis and generation on interrupt signal                                  |
| Background noises must not trigger interruption         | Three-layer defense: frontend amplitude threshold (55) + sustain window (500 ms), backend <code>min_interruption_duration</code> (0.5 s, LiveKit default), Silero VAD confidence threshold (0.7) |

Files Changed

| File                                                         | Change                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>frontend/src/hooks/useVoiceActivityMonitor.ts</code>   | Amplitude threshold raised to 55; sustain window raised to 500 ms                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <code>frontend/src/components/chat/InCallControls.tsx</code> | <code>handleInterruptAndSpeak()</code> , mic pre-warm effect, <code>greetingInProgress</code> guard on barge-in monitor                                                                                                                                                                                                                                                                                                                                                                      |
| <code>backend/src/taskorbit/livekit_agent/session.py</code>  | Originally added explicit <code>allow_interruptions=True / min_interruption_duration=0.8</code> plus Silero VAD params and <code>encoding="mp3_44100_128"</code> . The explicit interruption overrides were later <b>reverted</b> (#153, ambient noise/echo false-triggers) — current code leaves interruption handling at LiveKit defaults; Silero VAD params were retuned ( <code>deactivation_threshold</code> 0.45→0.6, <code>min_silence_duration</code> 0.55→1.5) and remain explicit. |
| <code>backend/tests/test_livekit_agent.py</code>             | Assertions updated to match the Silero VAD params and interruption behavior current at the time of each change                                                                                                                                                                                                                                                                                                                                                                               |

# Voice Call UI & Audio Pipeline Updates

## 1. Unified Audio Pipeline for Greeting and Agent Voice

**Problem:** The greeting message sounded different from the agent's voice during the call — different pitch, bitrate, and perceived quality.

**Root cause:** The greeting was played via a REST TTS call ( `/v1/tts/synthesize` ) returning `mp3_44100_128` , while the agent's responses came through the ElevenLabs LiveKit plugin defaulting to `mp3_22050_32` .

**Fix:**

- Routed the greeting through `session.say()` in `worker.py` so it travels the same WebRTC path as all agent turns.
- Set `encoding="mp3_44100_128"` explicitly on the `elevenlabs.TTS()` plugin so both paths use the same audio quality.
- Removed all frontend TTS overrides — the backend `.env` is now the single source of truth for voice config.

**Files:** `backend/src/taskorbit/worker.py` , `backend/src/taskorbit/livekit_agent/session.py`

## 2. Muted Status Indicator

**Problem:** When the mic was off mid-call ( `idle_in_call` ), nothing in the UI communicated that — it looked identical to an error state.

**Fix:** `CallStatusIndicator` now renders a `MicOff` icon with the label **"Muted"** when `status === "idle_in_call"` . The speaking label was also updated from "Responding..." to "Speaking..." for clarity.

**Files:** `frontend/src/components/chat/CallStatusIndicator.tsx`

## 3. Voice Button Mute/Unmute Toggle

**Problem:** Clicking the listening button while active did not physically stop the mic — it only changed visual state.

**Fix:** `handleVoiceBtnClick` in `InCallControls` now:

- Always calls `mic.disable()` when turning continuous mode off.
- Calls `mic.sendInterrupt()` if the agent is currently speaking, to immediately cut the TTS output.
- Sets the phase back to `idle_in_call` .

**Files:** `frontend/src/components/chat/InCallControls.tsx`

## 4. Mic Lock During Greeting

**Problem:** The mic could be open while the agent's greeting was playing, causing the user's voice (or ambient noise) to interrupt the greeting or be picked up prematurely.

**Fix:**

- A dedicated effect in `InCallControls` calls `mic.disable()` for the entire duration `greetingInProgress === true` .
- `useVoiceActivityMonitor` (barge-in detection) is gated on `!greetingInProgress` .
- The voice button is visually disabled during the greeting.

**Files:** `frontend/src/components/chat/InCallControls.tsx`

## 5. Automatic Continuous Mode After Greeting

**Problem:** After the greeting finished, the user had to manually press the mic button to start speaking.

**Fix:** When the greeting ends (first `speaking` → `idle_in_call` transition), `InCallControls` automatically enters continuous mode. A 15-second fallback in `ConversationalChat` releases the mic lock if the greeting never arrives (network delay, no greeting configured).

**Files:** `frontend/src/components/ConversationalChat.tsx` , `frontend/src/components/chat/InCallControls.tsx`

## 6. No-Speech Timeout (10 seconds)

**Problem:** If the agent was waiting for a response and the user said nothing, the mic stayed open in the listening state indefinitely.

**Fix:** A polling effect in `InCallControls` checks mic amplitude every 100 ms while `status === "recording"` . If the amplitude never crosses the speech floor ( `45` out of 255) for 10 continuous seconds, the mic is disabled, continuous mode is exited, and the phase returns to `idle_in_call` .

This is distinct from silence detection (which fires ~2 seconds after the user *starts* speaking and then pauses).

**Files:** `frontend/src/components/chat/InCallControls.tsx`

## 7. Noise Suppression — Three-Layer Defense

**Problem:** Background noises (keyboard clicks, ambient sounds) were triggering barge-in interruptions and false utterance sends.

**Fix applied at three layers:**

| Layer                                                         | Setting                             | Value                                                                         |
|---------------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------|
| Frontend barge-in<br>( <code>useVoiceActivityMonitor</code> ) | <code>SPEECH_THRESHOLD</code>       | 55 (up from 35)                                                               |
| Frontend barge-in<br>( <code>useVoiceActivityMonitor</code> ) | <code>SPEECH_SUSTAIN_MS</code>      | 500 ms (up from 300 ms)                                                       |
| Backend Silero VAD                                            | <code>activation_threshold</code>   | 0.7                                                                           |
| Backend Silero VAD                                            | <code>deactivation_threshold</code> | 0.6 (raised again, 0.45→0.6, for #153 — ambient fan noise held the turn open) |
| Backend Silero VAD                                            | <code>min_speech_duration</code>    | 0.2 s                                                                         |

|                    |                           |                                                                                                                                                                                                                      |
|--------------------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend Silero VAD | min_silence_duration      | 1.5 s (raised again, 0.55→1.5, held by the #102 regression guard)                                                                                                                                                    |
| Backend Silero VAD | prefix_padding_duration   | 0.4 s                                                                                                                                                                                                                |
| Backend LiveKit    | min_interruption_duration | Later <b>reverted</b> to the 0.5 s LiveKit default — a more aggressive 0.2 s/ <code>resume_false_interruption=False</code> override was tried but caused ambient noise/echo to cancel the agent's own answers (#153) |

**Files:** `frontend/src/hooks/useVoiceActivityMonitor.ts` , `backend/src/taskorbit/livekit_agent/session.py`

### 8. Post-Call Transcript — No Typing Animations

**Problem:** After ending a call, the transcript re-mounted fresh instances of `TranscriptBubble` . These new mounts saw `isFinal=true` turns and triggered a word-by-word reveal timer, causing text to type out letter-by-letter on the summary screen.

**Fix:** Added a `history` prop to `TranscriptBubble` . When `history=true` , the component bypasses all timer logic and renders the full text immediately. The post-call view passes `history` to every bubble.

**Files:** `frontend/src/components/history/TranscriptBubble.tsx` , `frontend/src/components/ConversationalChat.tsx`

### 9. Barge-in Mic Pre-Warm

**Problem:** When the user decided to interrupt the agent mid-speech, the first 300–500 ms of their words were always missed because enabling the mic cold-starts OS audio hardware.

**Fix:** When `status === "speaking"` and continuous mode is active, `InCallControls` pre-warms the mic via `mic.enable()` . The mic is now already publishing audio when the user decides to speak, so no words are lost. The backend's `allow_interruptions=True` handles the actual TTS cutoff.

**Files:** `frontend/src/components/chat/InCallControls.tsx`

### 10. Word-by-Word Streaming Transcript (Planned — Not Yet Deployed)

**Status:** Architecture designed and plan written; requires Docker rebuild to activate.

**Goal:** Agent transcript bubbles show words appearing one-by-one in sync with audio playback, rather than the full reply appearing all at once.

**Approach:**

- Backend: remove `sync_transcription=False` from `worker.py` so livekit-agents streams words progressively as TTS plays.
- Frontend: replace `reader.readAll()` with `for await` on the `TextStreamReader` in `useAgentTranscription.ts` .
- UI: add a blinking `█` cursor to the last active assistant bubble, driven by stream state (no timers).

**Files (pending):** `backend/src/taskorbit/worker.py` , `frontend/src/hooks/useAgentTranscription.ts` , `frontend/src/components/history/TranscriptBubble.tsx` , `frontend/src/components/ConversationalChat.tsx`

# Component Benchmarking

Issue: [#68 Component Benchmarking](#)

Branch: `Feature-Component-Benchmarking-68`

PR: [#184](#)

## Overview

Component benchmarking compares **STT / LLM / TTS** pipeline configurations on:

- Latency** — per-stage and end-to-end via `real /v1/conversations/process` API calls
- Tool-call reliability** — correct tool selected; for `data_extraction`, whether extracted values appear in the reply

**Scope:** text + voice paths.

| Path  | What is measured                                                    |
|-------|---------------------------------------------------------------------|
| text  | <code>llm_call</code> , <code>tool_call</code> , <code>total</code> |
| voice | Same plus STT (Deepgram) and TTS (Deepgram) provider timing         |

## Implementation status

| Area                                              | Status                |
|---------------------------------------------------|-----------------------|
| Benchmark harness                                 | ✅ Done                |
| 4 prompt categories (short/long, ± tools)         | ✅ Done                |
| 2 pipeline configs in YAML                        | ✅ Done                |
| Per-stage <code>latency_ms</code> on API response | ✅ Done                |
| Voice-path STT/TTS timing                         | ✅ Done                |
| Tool reliability layer                            | ✅ Done                |
| Aggregation + recommendation                      | ✅ Done                |
| <b>GPC standardized run</b>                       | ✅ <b>Completed</b>    |
| <b>Wiki results table</b>                         | ✅ <b>Filled below</b> |

## Pipeline configurations

| Label                                   | STT             | LLM                        | TTS                       |
|-----------------------------------------|-----------------|----------------------------|---------------------------|
| <code>oss-ollama-gemma4-deepgram</code> | Deepgram nova-2 | Ollama gemma4:26b (GCE VM) | Deepgram aura-2-andromeda |
| <code>oss-ollama-qwen3-deepgram</code>  | Deepgram nova-2 | Ollama qwen3.5:9b (GCE VM) | Deepgram aura-2-andromeda |

Ollama runs on a team GCE VM at `http://35.231.129.211:11434/`. Before timed runs the harness primes the model into VRAM with a configurable warmup + 30s settle buffer.

## Benchmark results

### GPC standardized run

|                     |                                                               |
|---------------------|---------------------------------------------------------------|
|                     |                                                               |
| <b>Date</b>         | 2026-07-01                                                    |
| <b>Environment</b>  | Local backend ( <code>localhost:8000</code> ) + Ollama GCE VM |
| <b>Repetitions</b>  | 1 per config/path                                             |
| <b>Path</b>         | text + voice                                                  |
| <b>Total rows</b>   | 48                                                            |
| <b>Success rate</b> | <b>100%</b> (48/48)                                           |

### Headline comparison

| Config                                  | Avg total text (ms) | Avg total voice (ms) | Reliability    |
|-----------------------------------------|---------------------|----------------------|----------------|
| <code>oss-ollama-gemma4-deepgram</code> | <b>4,682</b>        | <b>4,658</b>         | <b>91.7% ★</b> |
| <code>oss-ollama-qwen3-deepgram</code>  | 4,851               | 4,983                | 91.7%          |

**Recommended default:** `oss-ollama-gemma4-deepgram`

**Reason:** Lowest avg total latency (text: 4,682 ms, voice: 4,658 ms); reliability tied at 91.7%.

### Per-stage averages (text)

| Config                                  | <code>llm_call</code> (ms) | <code>tool_call</code> (ms) | <code>total</code> (ms) |
|-----------------------------------------|----------------------------|-----------------------------|-------------------------|
| <code>oss-ollama-gemma4-deepgram</code> | 1,607                      | 0.1                         | 4,682                   |
| <code>oss-ollama-qwen3-deepgram</code>  | 1,761                      | 0.2                         | 4,851                   |

### Per-stage averages (voice)

| Config                     | llm_call (ms) | tool_call (ms) | total (ms) |
|----------------------------|---------------|----------------|------------|
| oss-ollama-gemma4-deepgram | 1,569         | 1.0            | 4,658      |
| oss-ollama-qwen3-deepgram  | 1,908         | 0.1            | 4,983      |

Latency by category (text)

| Category        | gemma4 (ms) | qwen3.5 (ms) |
|-----------------|-------------|--------------|
| short_no_tool   | 3,275       | 3,358        |
| short_with_tool | 4,480       | 4,676        |
| long_no_tool    | 4,864       | 5,045        |
| long_with_tool  | 5,175       | 5,366        |

Latency by category (voice)

| Category        | gemma4 (ms) | qwen3.5 (ms) |
|-----------------|-------------|--------------|
| short_no_tool   | 3,185       | 3,377        |
| short_with_tool | 4,473       | 4,170        |
| long_no_tool    | 4,741       | 5,402        |
| long_with_tool  | 5,235       | 5,537        |

Reliability by category

| Category        | gemma4 | qwen3.5 |
|-----------------|--------|---------|
| short_no_tool   | 100%   | 100%    |
| short_with_tool | 100%   | 100%    |
| long_no_tool    | 100%   | 100%    |
| long_with_tool  | 80%    | 80%     |

Both models show the same reliability pattern: 100% on all categories except `long_with_tool` (appointment booking with `data_extraction` ), where one of five prompts failed to incorporate extracted values into the reply.

Results file

```
benchmarks/results/gpc-run-20260702T004141/
├─ component/
│ └─ 2026-07-01T22-41-41_component_benchmark.jsonl # raw 48 rows
├─ component-benchmark-report.txt # full text report
├─ index.csv # per-config/path summary
├─ benchmark-summary.csv # Excel-friendly export
└─ run.log # harness output
```

Repo map

| Path                                                 | Purpose                                       |
|------------------------------------------------------|-----------------------------------------------|
| benchmarks/runner/run_component_benchmark.py         | CLI entry point                               |
| benchmarks/runner/component_runner.py                | Multi-turn API driver (text + voice paths)    |
| benchmarks/runner/voice_stages.py                    | Direct STT/TTS provider timing for voice rows |
| benchmarks/runner/ollama_warmup.py                   | VRAM primer + settle buffer                   |
| benchmarks/runner/turn_expectations.py               | Per-turn tool/status expectations             |
| benchmarks/runner/reliability.py                     | Authoritative reliability verdicts            |
| benchmarks/runner/aggregate.py                       | JSONL → index.csv + report                    |
| benchmarks/runner/recommendation.py                  | Default config picker                         |
| benchmarks/configs/component-benchmark.yaml          | Reference config matrix                       |
| benchmarks/configs/component-benchmark-oss-only.yaml | OSS-only config (used for GPC run)            |
| benchmarks/scripts/run-gpc-benchmark.sh              | GPC one-shot script                           |

Changelog

| Date       | Change                                                                                |
|------------|---------------------------------------------------------------------------------------|
| 2026-07-01 | GPC standardized run completed — 48/48 success (gemma4:26b vs qwen3.5:9b, text+voice) |
| 2026-07-01 | Ollama VRAM warmup + 30s buffer before timed benchmark runs                           |
| 2026-07-01 | Voice path restored in standard config                                                |

|            |                                                                              |
|------------|------------------------------------------------------------------------------|
| 2026-06-30 | Initial implementation; text smoke run (failed — OpenAI 429/Ollama timeouts) |
|------------|------------------------------------------------------------------------------|

---

# Logging & Observability

## Logging & Observability

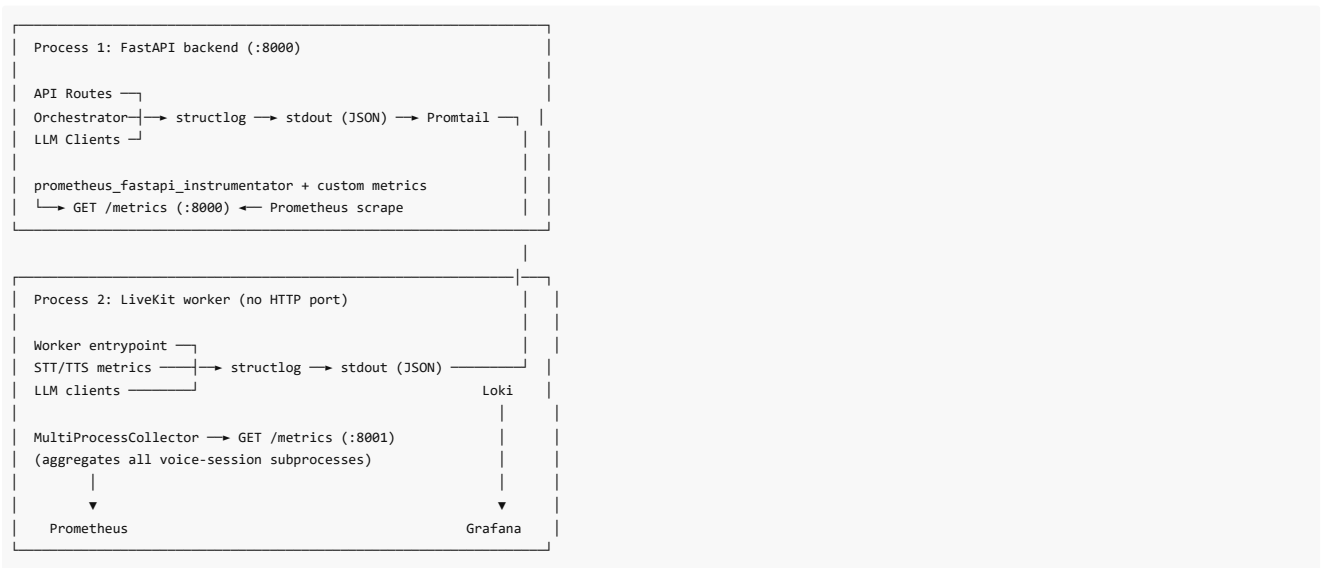
This page documents the structured logging and observability infrastructure for the TaskOrbit backend. It covers how logs and metrics are generated by each component and aggregated centrally for debugging and performance monitoring.

### Table of Contents

- [Architecture Overview](#)
- [Structured Logging](#)
- [Log Format](#)
- [Component Log Coverage](#)
- [Pipeline Latency Measurement](#)
- [Metrics \(Prometheus\)](#)
- [Log Aggregation \(Loki + Promtail\)](#)
- [Grafana Dashboards](#)
- [Trace ID Correlation](#)
- [Configuration Reference](#)
- [Adding Logging to a New Component](#)

### Architecture Overview

TaskOrbit runs as **two separate processes** — the FastAPI backend and the LiveKit voice worker — each with its own Prometheus metrics server and log stream.



Every backend component emits **structured logs** (key-value pairs with timestamp and severity level) to stdout. Promtail ships those logs to Loki. Prometheus scrapes both `/metrics` endpoints (`:8000` and `:8001`). Grafana provides unified dashboards over both.

**Voice path:** when a user speaks through the microphone, the browser sends audio over LiveKit WebRTC only — **no HTTP calls** are made to `/v1/conversations/process` or `/v1/tts/synthesize`. All STT → LLM → TTS processing happens inside the worker process. This is why REST-path Prometheus metrics do not capture voice turns; Loki `Llm_call_completed` log events are used instead for unified token tracking across both paths.

### Structured Logging

The backend uses [structlog](#) for all structured logging, configured in `backend/src/taskorbit/logging/setup.py`.

#### Setup

`configure_logging()` is called once at process start — in `create_app()` for the FastAPI process and in `run_worker()` for the LiveKit worker. It is idempotent.

```
from taskorbit.logging.setup import configure_logging, get_logger

configure_logging() # once at startup
logger = get_logger(__name__) # per module
```

#### Using the logger

```
logger.info("intent_detected", intent="book_service", conversation_id="abc-123")
logger.warning("worker_generate_reply_failed", error=str(exc))
logger.error("tts_elevenlabs_error", status=502, tts_latency_ms=1240.5)
logger.debug("stt_transcript_received", length=42)
```

Every call produces a single log line. Additional key-value pairs become searchable fields in Loki.

**Rule:** Always use `from taskorbit.Logging.setup import get_logger`. Never use `import logging / logging.getLogger` or `structlog.get_logger()` directly — those bypass the configured processor chain and will not carry structured context in production.

Log Format

The renderer is selected automatically based on `APP_ENV` :

| Environment          | Format                             | Purpose                             |
|----------------------|------------------------------------|-------------------------------------|
| development          | Coloured console (ConsoleRenderer) | Human-readable in terminal          |
| staging / production | JSON (JSONRenderer)                | Machine-parseable, ingested by Loki |

Example — development (console)

```
2026-05-19T01:23:45Z [info] llm_call_completed provider=openai model=gpt-4o-mini chars=312 llm_api_latency_ms=843.2
```

Example — production (JSON)

```
{
 "timestamp": "2026-05-19T01:23:45.123456Z",
 "level": "info",
 "event": "llm_call_completed",
 "provider": "openai",
 "model": "gpt-4o-mini",
 "chars": 312,
 "llm_api_latency_ms": 843.2,
 "trace_id": "a1b2c3d4e5f6",
 "span_id": "f6e5d4c3"
}
```

Every log line includes:

- `timestamp` — UTC ISO-8601
- `level` — `debug` / `info` / `warning` / `error`
- `event` — snake\_case event name (the first positional argument)
- `trace_id` / `span_id` — injected automatically by `TraceIDMiddleware` for HTTP requests

Component Log Coverage

| Component            | Module                            | Key log events                                                                                                                                                                                                                |
|----------------------|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FastAPI startup      | api/main.py                       | api_starting, api_shutdown                                                                                                                                                                                                    |
| Conversations route  | api/routes/conversations.py       | conversation_request_received, conversation_request_completed, orchestration_not_implemented, conversation_created, messages_persisted, failed_to_save_user_message, failed_to_save_assistant_message, conversation_not_found |
| LiveKit token route  | api/routes/livekit.py             | token minting events                                                                                                                                                                                                          |
| TTS route            | api/routes/tts.py                 | tts_synthesize_ok, tts_elevenlabs_error                                                                                                                                                                                       |
| Orchestrator         | orchestration/__init__.py         | intent_detected, agent_selected, pipeline_complete, llm_config_error, llm_timeout, invalid_runtime_input, runtime_error                                                                                                       |
| OpenAI client        | integrations/llm/openai_client.py | llm_call_started, llm_call_completed (with prompt_tokens, completion_tokens), llm_call_failed                                                                                                                                 |
| Gemini client        | integrations/llm/gemini_client.py | llm_call_started, llm_call_completed (with prompt_tokens, completion_tokens), llm_call_failed                                                                                                                                 |
| LiveKit agent        | livekit_agent/llm.py              | stt_transcript_received (length only — content never logged)                                                                                                                                                                  |
| Worker               | worker.py                         | worker_generate_reply_triggered, worker_generate_reply_failed, worker_interrupt_requested, worker_interrupt_failed, worker_greeting_spoken, worker_metrics_server_started, tts_metrics_collected, stt_metrics_collected       |
| Database CRUD        | database/crud.py                  | get_user_failed, create_conversation_message_failed, etc.                                                                                                                                                                     |
| Metrics module       | observability/metrics.py          | default_metrics_registered, default_metrics_already_registered, metrics_labels_initialized                                                                                                                                    |
| Unhandled exceptions | api/main.py (exception handler)   | unhandled_error                                                                                                                                                                                                               |

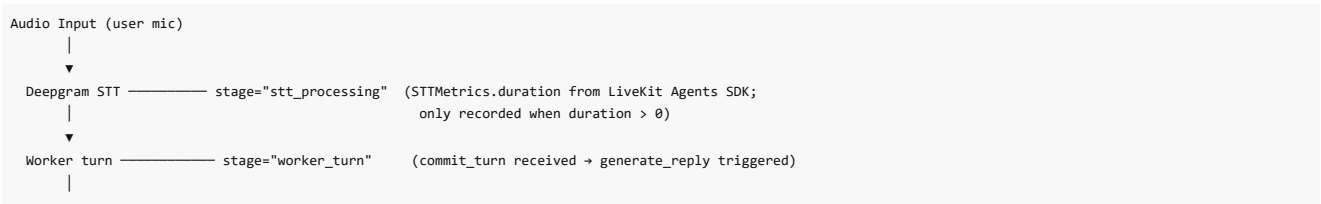
`llm_call_completed` is the key event for token tracking. Both the OpenAI and Gemini clients emit this event with `prompt_tokens` and `completion_tokens` fields on every successful LLM call — from both the text (REST) and voice (worker) paths. Grafana token panels use `Loki LogQL unwrap` queries over this event to aggregate totals across both paths.

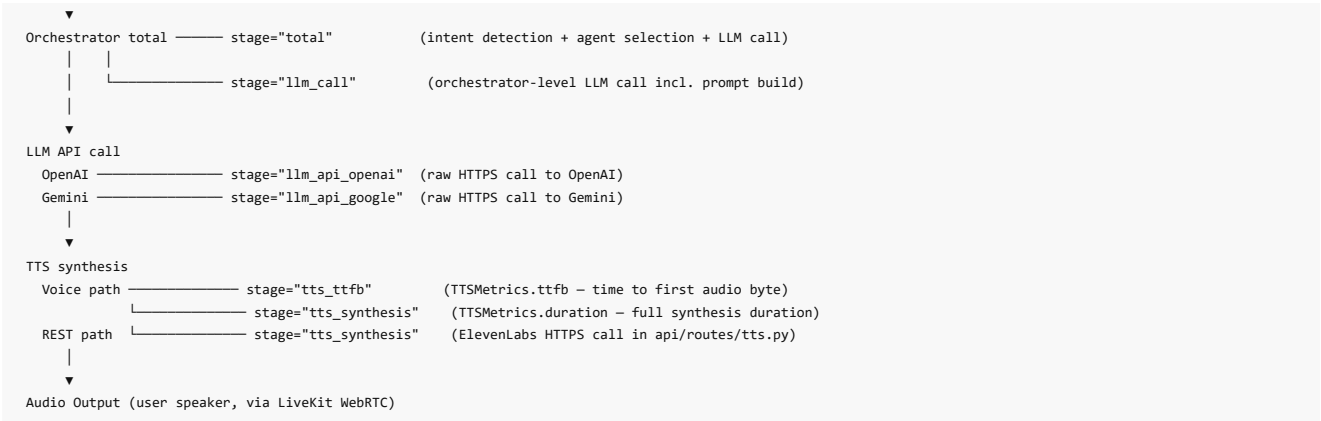
**Privacy note:** STT transcript content is never logged. Only `Length=<n>` is emitted at debug level.

Pipeline Latency Measurement

Request duration is measured at every stage of the voice pipeline using `time.perf_counter()` . Each measurement is recorded both as a **Prometheus histogram** and as a **millisecond field** in the structured log line.

Pipeline stages





**Note on `tts_synthesis`** : both the voice path (worker `_on_metrics TTSMetrics`) and the REST path ( `/v1/tts/synthesize` ) observe the same `tts_synthesis` stage. In a pure voice deployment, REST TTS calls are rare (greeting only), so the histogram values are predominantly voice calls. The `tts_ttfb` stage is voice-only.

### Histogram buckets

All stages use the same `taskorbit_pipeline_latency_seconds` histogram with buckets: `0.05s`, `0.1s`, `0.25s`, `0.5s`, `1.0s`, `2.5s`, `5.0s`, `10.0s`

### Log fields per component

| Component            | Log field                                          | Unit      |
|----------------------|----------------------------------------------------|-----------|
| Worker (commit_turn) | turn_latency_ms                                    | ms        |
| Worker (TTSMetrics)  | ttfb_ms, duration_ms, audio_duration_ms, cancelled | ms / bool |
| Worker (STTMetrics)  | duration_ms, audio_duration_ms                     | ms        |
| Orchestrator         | llm_latency_ms, total_latency_ms                   | ms        |
| OpenAI client        | llm_api_latency_ms                                 | ms        |
| Gemini client        | llm_api_latency_ms                                 | ms        |
| TTS route            | tts_latency_ms                                     | ms        |

### Metrics (Prometheus)

Custom metrics are defined in `backend/src/taskorbit/observability/metrics.py` and exposed at `GET /metrics` (controlled by `METRICS_ENABLED=true` ).

### Custom metrics

| Metric                                               | Type      | Labels                                                               | Description                                                                                                                                                                          |
|------------------------------------------------------|-----------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>taskorbit_llm_requests_total</code>            | Counter   | <code>provider</code> , <code>model</code> , <code>status</code>     | Total LLM API calls by provider, model, and outcome                                                                                                                                  |
| <code>taskorbit_llm_response_chars</code>            | Histogram | <code>provider</code> , <code>model</code>                           | LLM response size in characters                                                                                                                                                      |
| <code>taskorbit_pipeline_latency_seconds</code>      | Histogram | <code>stage</code>                                                   | Latency per named pipeline stage (see above)                                                                                                                                         |
| <code>taskorbit_conversation_errors_total</code>     | Counter   | <code>error_type</code>                                              | Errors during conversation processing ( <code>llm_timeout</code> , <code>encoding_error</code> , <code>invalid_input</code> , <code>runtime_error</code> , <code>llm_config</code> ) |
| <code>taskorbit_tokens_used_total</code>             | Counter   | <code>provider</code> , <code>model</code> , <code>token_type</code> | Token consumption (prompt + completion) — Prometheus only; see Loki note below                                                                                                       |
| <code>taskorbit_voice_pipeline_requests_total</code> | Counter   | <code>handler</code> , <code>status</code>                           | Voice pipeline requests processed (handler values: <code>/v1/tts/synthesize</code> , <code>/v1/conversations/process</code> )                                                        |
| <code>taskorbit_voice_turn_latency_seconds</code>    | Histogram | (none)                                                               | End-to-end voice turn latency from <code>commit_turn</code> to reply text ready                                                                                                      |

**LLm\_config error type:** the orchestrator has a dedicated `except LLMConfigError` block that increments `conversation_errors_total{error_type="LLm_config"}` and logs `LLm_config_error` with hint="Check that the provider API key is set in `.env`". This makes missing API key misconfiguration immediately visible in the Grafana Conversation Error Rate panel.

**Token metrics and the voice path:** `taskorbit_tokens_used_total` is incremented inside the LLM clients which run in voice-session **subprocesses** forked by the LiveKit Agents SDK. Without multiprocess Prometheus mode, those increments are invisible to the worker's HTTP server. See the **Worker Metrics Server** section below.

### Default system metrics

`configure_default_metrics()` registers the Python equivalents of Node.js `collectDefaultMetrics` :

| Collector                      | Metrics provided                                                                         |
|--------------------------------|------------------------------------------------------------------------------------------|
| <code>ProcessCollector</code>  | CPU seconds, RSS/virtual memory, open file descriptors, start time ( <i>Linux only</i> ) |
| <code>GCCollector</code>       | Garbage collection counts and durations by generation                                    |
| <code>PlatformCollector</code> | Python version info                                                                      |

### HTTP metrics (auto-instrumented)

`prometheus_fastapi_instrumentator` automatically instruments all HTTP routes:

| Metric | Description |
|--------|-------------|
|--------|-------------|

|                               |                                               |
|-------------------------------|-----------------------------------------------|
| http_requests_total           | Request count by handler, method, status code |
| http_request_duration_seconds | Request latency histogram by handler          |
| http_requests_in_progress     | Concurrent in-flight requests                 |

The `/health` endpoint is excluded from HTTP metrics to avoid noise.

### Worker Metrics Server

The LiveKit worker process exposes its own Prometheus endpoint at `:8001/metrics`, scraped separately from the backend's `:8000/metrics`.

#### Multiprocess mode (critical)

LiveKit Agents v1.x forks a **subprocess** for each voice session. Without multiprocess mode, `tokens_used_total.inc()` calls inside the subprocess update only that subprocess's in-memory registry — invisible to the main process's HTTP server on port 8001.

**Fix:** set `PROMETHEUS_MULTIPROC_DIR` to a shared directory and use `MultiProcessCollector` in the metrics HTTP server.

```
docker-compose.yml (already configured)
worker:
 environment:
 PROMETHEUS_MULTIPROC_DIR: /tmp/prometheus_multiproc
 volumes:
 - taskorbit-worker-prometheus:/tmp/prometheus_multiproc
```

`run_worker()` in `worker.py` detects `PROMETHEUS_MULTIPROC_DIR` and automatically switches to `MultiProcessCollector`:

```
if multiproc_dir:
 registry = CollectorRegistry()
 MultiProcessCollector(registry)
 start_http_server(port=8001, registry=registry)
else:
 start_http_server(port=8001)
```

**Never remove `PROMETHEUS_MULTIPROC_DIR` from the worker environment** — doing so silently breaks all voice-turn token and latency metrics in Prometheus.

#### Production scraping (Cloud Run)

In production, Prometheus scrapes both the backend and worker via their public HTTPS Cloud Run URLs — no auth token is required because both services have `allUsers` as `roles/run.invoker`. The scrape config stored in Secret Manager (`taskorbit-prometheus-config`) uses `scheme: https` and the host extracted from the Cloud Run URL.

Do **not** add `credentials_file: /var/run/secrets/google/token` to the scrape config — that path is GKE-specific and does not exist on Cloud Run.

The worker was originally configured with `INGRESS_TRAFFIC_INTERNAL_ONLY`, which blocked Prometheus from reaching port 8001. It now uses `INGRESS_TRAFFIC_ALL` so Prometheus can scrape it via the public URL. See `docs/Cloud-Deployment.md` for the full ingress policy table.

#### Why Grafana token panels use Loki instead of Prometheus

Combining `taskorbit_tokens_used_total` from both the backend (`:8000`) and worker (`:8001`) Prometheus jobs in a single Grafana stat panel requires two separate queries and manual addition. Loki is simpler: both processes log `llm_call_completed` with `prompt_tokens / completion_tokens` to stdout, Promtail ships all container logs to Loki, and a single LogQL query covers both text and voice turns:

```
Total prompt tokens today (text + voice)
sum_over_time(
 {service=~"backend|worker"} | json | event = "llm_call_completed"
 | unwrap prompt_tokens [$__range]
)
```

**Do not switch the token panels back to Prometheus** — a single Prometheus stat value cannot natively sum across two jobs without recording rules.

### Log Aggregation (Loki + Promtail)

All container stdout is shipped to Loki automatically. No code changes are needed when adding new components.

#### Local development (Docker Compose)

- Every backend process writes JSON logs to stdout
- Promtail reads Docker container stdout via the Docker socket (`/var/run/docker.sock`)
- Promtail parses the `level` and `event` JSON fields and promotes them to Loki labels
- Logs are pushed to Loki at `http://loki:3100/loki/api/v1/push`
- Grafana queries Loki for log exploration and alerting

#### Production (Cloud Run)

There is no Docker daemon on Cloud Run, so Promtail cannot mount `/var/run/docker.sock`. The production pipeline is:

- Cloud Run writes container stdout → **Cloud Logging** (automatic, no config needed)
- A Cloud Logging sink (`taskorbit-loki-sink`) filters `taskorbit-backend` and `taskorbit-worker` log entries
- The sink publishes matching entries to the `taskorbit-loki-logs` Pub/Sub topic
- A Pub/Sub push subscription delivers messages to Loki's HTTP push endpoint with an OIDC token for auth

**Known limitation:** Pub/Sub wraps each log entry in a base64 envelope; Loki's `/loki/api/v1/push` expects its own stream JSON format. Direct push may not parse correctly. The proper fix is a Promtail Cloud Run service configured with a `gcplog` Pub/Sub input source, which handles the envelope and forwards correctly-formatted entries to Loki.

#### Loki labels

| Label | Source | Example values |
|-------|--------|----------------|
|-------|--------|----------------|

|           |                             |                                     |
|-----------|-----------------------------|-------------------------------------|
| container | Docker container name       | taskorbit-backend, taskorbit-worker |
| service   | Docker Compose service name | backend, worker                     |
| level     | JSON level field            | info, warning, error                |

#### Useful LogQL queries

```
All errors from the backend (text path only)
{service="backend"} | level = "error"

All errors across both backend and voice worker
{service=~"backend|worker"} | level = "error"

All logs for a specific conversation
{service="backend"} |= "conversation_id" | json | conversation_id = "abc-123"

LLM call latency above 2 seconds (both paths)
{service=~"backend|worker"} | json | event = "llm_call_completed" | llm_api_latency_ms > 2000

Filter by trace ID (HTTP request correlation)
{service="backend"} |= "a1b2c3d4e5f6"

TTS errors in the last hour
{service="backend"} | json | event = "tts_elevenlabs_error"

Total prompt tokens today (text + voice combined)
sum_over_time(
 {service=~"backend|worker"} | json | event = "llm_call_completed"
 | unwrap prompt_tokens [1d]
)

Token consumption rate per minute (text + voice)
rate(
 {service=~"backend|worker"} | json | event = "llm_call_completed"
 | unwrap prompt_tokens [1m]
)

Voice worker STT/TTS timing events
{service="worker"} | json | event =~ "tts_metrics_collected|stt_metrics_collected"

LLM config errors (missing API key)
{service=~"backend|worker"} | json | event = "llm_config_error"
```

#### Grafana Dashboards

Grafana is available at <http://localhost:3000> (anonymous admin in dev). The **TaskOrbit FastAPI Observability** dashboard ( `uid: taskorbit-overview` ) is auto-provisioned on first boot — no manual setup required.

#### Dashboard template variables

| Variable      | Type                 | Purpose                                                               |
|---------------|----------------------|-----------------------------------------------------------------------|
| \$application | Query (label_values) | Filter all Prometheus panels by job label; default: taskorbit-backend |
| \$log_query   | Textbox              | Appended to the Loki query; e.g. <code>  level = "error"</code>       |

#### Dashboard panels

| Row              | Panel                       | Data source | Notes                                      |
|------------------|-----------------------------|-------------|--------------------------------------------|
| (top)            | Total Requests (24h)        | Prometheus  |                                            |
| (top)            | Requests Count per endpoint | Prometheus  |                                            |
| (top)            | Requests Average Duration   | Prometheus  |                                            |
| (top)            | Total Exceptions / 5xx      | Prometheus  |                                            |
| (top)            | Percent of 2xx Requests     | Prometheus  |                                            |
| (top)            | Percent of 5xx Requests     | Prometheus  |                                            |
| (top)            | P99 Request Duration        | Prometheus  |                                            |
| (top)            | Requests In Progress        | Prometheus  |                                            |
| (top)            | Request Per Sec             | Prometheus  |                                            |
| (top)            | Log Type Rate (by level)    | Loki        | CRITICAL / ERROR / WARNING / INFO coloured |
| (top)            | Log of All FastAPI App      | Loki        | Filtered by \$application + \$log_query    |
| HTTP Performance | HTTP Request Rate           | Prometheus  |                                            |

|                  |                                   |            |                                                                                                                                          |
|------------------|-----------------------------------|------------|------------------------------------------------------------------------------------------------------------------------------------------|
| HTTP Performance | HTTP P95 Latency                  | Prometheus |                                                                                                                                          |
| LLM & Pipeline   | Total Tokens Used Today           | Loki       | sum_over_time + unwrap prompt_tokens/completion_tokens [ \$__range ] over {service=~"backend worker"} — covers both text and voice turns |
| LLM & Pipeline   | LLM Requests by Provider / Status | Prometheus |                                                                                                                                          |
| LLM & Pipeline   | LLM Response Size P95             | Prometheus |                                                                                                                                          |
| LLM & Pipeline   | Token Consumption Rate            | Loki       | rate() + unwrap prompt_tokens — real-time across both paths                                                                              |
| LLM & Pipeline   | Cumulative Tokens Today           | Loki       | sum_over_time + unwrap [id]                                                                                                              |
| LLM & Pipeline   | Pipeline Latency P50 / P95 / P99  | Prometheus |                                                                                                                                          |
| LLM & Pipeline   | End-to-End Pipeline Latency P95   | Prometheus |                                                                                                                                          |
| LLM & Pipeline   | Conversation Error Rate by Type   | Prometheus |                                                                                                                                          |
| Logs & Traces    | Backend Logs (ERROR + WARNING)    | Loki       |                                                                                                                                          |
| Logs & Traces    | Request Trace Log Explorer        | Loki       | Filter by trace_id                                                                                                                       |

Token panels use **Loki**, not **Prometheus**. See the [Worker Metrics Server](#) section for why. Do not switch them back to Prometheus.

New HTTP routes are picked up automatically via `by(handler)` label grouping — no dashboard changes needed.

### Trace ID Correlation

`TraceIDMiddleware` ( `observability/middleware.py` ) runs on every HTTP request:

1. Reads the incoming `X-Trace-Id` header, or generates a new UUID if absent
2. Returns the trace ID in the response `X-Trace-Id` header
3. Binds `trace_id` and `span_id` into structlog contextvars

Every log line emitted during that request automatically carries `trace_id` and `span_id`. This lets you filter all logs for a single request in Loki:

```
{service="backend"} |= "a1b2c3d4e5f6"
```

**Note:** Background tasks spawned via `asyncio.create_task` do **not** inherit the HTTP request context automatically. Copy the context manually if trace correlation is needed inside a background task.

### Configuration Reference

All variables are declared in `backend/.env.example`.

| Variable                    | Default     | Effect                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| APP_ENV                     | development | development → console logs; any other value → JSON logs                                                                                                                                                                                                                                                                                             |
| LOG_LEVEL                   | INFO        | Minimum log level: DEBUG, INFO, WARNING, ERROR                                                                                                                                                                                                                                                                                                      |
| METRICS_ENABLED             | true        | Exposes GET <code>/metrics</code> Prometheus endpoint on the FastAPI process                                                                                                                                                                                                                                                                        |
| OTEL_ENABLED                | false       | Initialises OpenTelemetry SDK (stdout exporter if no endpoint set)                                                                                                                                                                                                                                                                                  |
| OTEL_EXPORTER_OTLP_ENDPOINT | (empty)     | gRPC OTLP endpoint, e.g. <code>http://otel-collector:4317</code>                                                                                                                                                                                                                                                                                    |
| PROMETHEUS_MULTIPROC_DIR    | (empty)     | When set, enables <code>prometheus_client</code> multiprocess mode — <b>required for the LiveKit worker</b> so voice-session subprocess metrics are visible at <code>:8001/metrics</code> . Set to <code>/tmp/prometheus_multiproc</code> in <code>docker-compose.yml</code> for the worker service. Never remove this from the worker environment. |

### Docker Compose service ports

| Service        | Port | Purpose                                                                                         |
|----------------|------|-------------------------------------------------------------------------------------------------|
| prometheus     | 9090 | Scrapes <code>backend:8000/metrics</code> and <code>worker:8001/metrics</code> every <b>5 s</b> |
| loki           | 3100 | Log aggregation API (no auth — local dev only)                                                  |
| grafana        | 3000 | Dashboards (anonymous admin — local dev only)                                                   |
| promtail       | —    | Sidecar; no host port                                                                           |
| worker metrics | 8001 | Prometheus endpoint for the LiveKit worker process (not exposed to host)                        |

### Adding Logging to a New Component

1. **Import the logger** at the top of the module:

```
from taskorbit.logging.setup import get_logger
logger = get_logger(__name__)
```

2. **Log meaningful events** with structured key-value context:

```
logger.info("my_event_name", key1="value", key2=42)
logger.error("my_error_name", error=str(exc), detail="what went wrong")
```

3. **Measure latency** if the component makes network calls or has meaningful duration:

```
import time
from taskorbit.observability.metrics import get_metrics

_start = time.perf_counter()
... do work ...
_elapsed = time.perf_counter() - _start

get_metrics().pipeline_latency_seconds.labels(stage="my_stage").observe(_elapsed)
logger.info("my_event_completed", latency_ms=round(_elapsed * 1000, 1))
```

4. **To add a new LLM provider**, follow the pattern in `openai_client.py` and `gemini_client.py` :

- Call `get_metrics().llm_requests_total.labels(provider=..., model=..., status=...).inc()` in every `except` block and after the success log line
- Call `.llm_response_chars.labels(...).observe(len(text))` on success
- Call `.tokens_used_total.labels(..., token_type="prompt").inc(n)` and `token_type="completion"` on success
- Call `.pipeline_latency_seconds.labels(stage="llm_api_<provider>").observe(elapsed)` around the raw HTTPS call
- Pre-initialize the new provider's label combinations in `configure_default_metrics()` in `metrics.py`

5. **No registration needed** — Promtail picks up new containers automatically, and new `stage=` labels appear in Grafana's pipeline latency panel immediately.